Enhancing Breast Cancer Diagnosis with Explainable Machine Learning Approaches

Marlene Fuhr
Rice University
mf88@rice.edu

ACM Reference Format:
Marlene Fuhr, Rice University, mf88@rice.edu. 2025. Enhancing Breast Cancer Diagnosis with Explainable Machine Learning Approaches. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Breast cancer is the most common malignancy among women globally, representing nearly one in four cancer cases in females and causing approximately 685,000 deaths annually worldwide . In 2018 alone, 2.1 million women were diagnosed with breast cancer, making it a leading contributor to cancer-related mortality in over 100 countries . Early and accurate diagnosis significantly increases survival rates, yet the manual interpretation of mammograms is prone to human error, often resulting in missed diagnoses or false positives that lead to unnecessary biopsies and patient anxiety . For every 1,000 women screened, approximately 100 are recalled, and 35 undergo biopsies, with many of these procedures proving unnecessary .

The challenge is compounded in developing countries, where limited access to healthcare, lack of awareness, and late stage presentation contribute to high mortality rates. For instance, in India, breast cancer accounts for 27% of all cancers, with a lifetime risk of 1 in 22 for urban women . In low resource settings, traditional diagnostic methods like manual mammography review are often insufficient to meet the growing need for timely and reliable screening.

To address these challenges, machine learning (ML) and deep learning (DL) approaches have emerged as powerful tools for automating and enhancing breast cancer detection. These technologies not only reduce the burden on radiologists but also offer potential improvements in diagnostic accuracy, speed, and reproducibility. For example, CNN-based classifiers have achieved up to 96.7% precision in distinguishing between benign and malignant tumors on standardized datasets . Moreover, ML models like support vector machines and ensemble classifiers have shown robust performance with accuracies exceeding 94% on preprocessed mammographic data .

Despite these advances, current systems still face critical limitations, including class imbalance, lack of standardization in image preprocessing, and insufficient explainability of model predictions, which restrict their broader clinical adoption. This project contributes by integrating feature selection, model interpretability, and robust evaluation, with the goal of enhancing diagnostic support systems. Developing accurate and interpretable breast cancer classifiers not only aids clinical decision-making but also has the potential to save thousands of lives annually by enabling earlier intervention and more personalized treatment plans.

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 RELATED WORK

Numerous studies have applied machine learning to breast cancer classification, particularly using mammographic images. Kourou et al. provided an extensive review of ML techniques for cancer prognosis, highlighting the promise of feature selection and ensemble models for improving prediction performance . In another study, Sharma et al. used patient biopsy samples from the Malwa region in India and applied machine learning classifiers with Weka to achieve an accuracy of 90.47% in early cancer detection .

Deep learning approaches have also shown effectiveness, especially in image segmentation and classification tasks. Abdelhafiz et al. evaluated convolutional neural networks (CNNs) for mammogram segmentation and discussed the challenges associated with limited data and model interpretability . Similarly, Wang et al. proposed the use of Extreme Learning Machines (ELM) with textural and morphological features, achieving competitive results compared to traditional SVMs .

However, while deep learning excels in performance, simpler models like logistic regression remain attractive for their interpretability, especially when combined with feature selection methods such as Sequential Backward Selection (SBS). This report builds on these insights by combining classical ML techniques with modern data balancing and feature selection strategies to improve classification accuracy and transparency.

3 METHODS

3.1 Summary of Work

My methodology starts with extracting mammogram images from the MIAS dataset. I did image preprocessing, denoising, CLAHE for contrast enhancement, and segmentation with Hough Transformation to remove the pectoral muscle. We extract features such as contrast, homogeneity, standard deviation and skewness of intensity, eccentricity, radius, background tissue type, and class of abnormality. The features are compiled into a CSV file.

I then scale the features, handle class imbalance using SMOTE, and apply stratified splitting into train, validation, and test setsin

Marlene Fuhr
Rice University
mf88@rice.edu

60%,20% and 20% respectively. Logistic regression is used as a baseline model, followed by support vector machines with kernel optimization, random forests with hyperparameter tuning, and neural networks. For model interpretability and dimensionality reduction, I apply Sequential Backward Selection (SBS). All models are evaluated using accuracy, F1-score, training/validation/test error, 5-fold cross-validation mean accuracy and standard deviation. This thorough approach ensures fair comparison and reliability of findings.

3.2 Data

I worked with a total of 108 mammogram images extracted from the MIAS (Mammographic Image Analysis Society) database. From these images, I manually extracted 14 relevant features based on domain literature and prior studies. The dataset is inherently imbalanced, with class labels representing severity of condition: approximately 67% of the images correspond to normal cases (41 samples), 19% to benign cases (11 samples), and 13% to malignant cases (8 samples). For modeling purposes, I applied label encoding to define the target variable `Severity`, where 0 represents normal, 1 represents benign, and 2 represents malignant findings.

In addition to severity, the dataset includes categorical features such as background tissue type and class of abnormality. Background tissue type was encoded as Fatty (F), Fatty-glandular (G), and Dense-glandular (D). The class of abnormality included categories like CALC, CIRC, SPIC, MISC, ARCH, ASYM, and NORM. For preprocessing, I handled missing data in the `radius of circle` feature by imputing a value of zero. This decision was based on the clinical assumption that missing radius values indicate that no lesion was present, hence the absence of a measurable circular abnormality.

To understand the relationships between features, I computed a Pearson correlation heatmap. The analysis revealed several strongly correlated features. Among the most significant were `radius of circle` (correlation = 0.77), `class of abnormality_ASYM` (correlation = 0.65), and `class of abnormality_NORM` (correlation = -0.92). These correlations informed feature selection and model design. The heatmap visualization in Figure 1 demonstrates these relationships clearly.

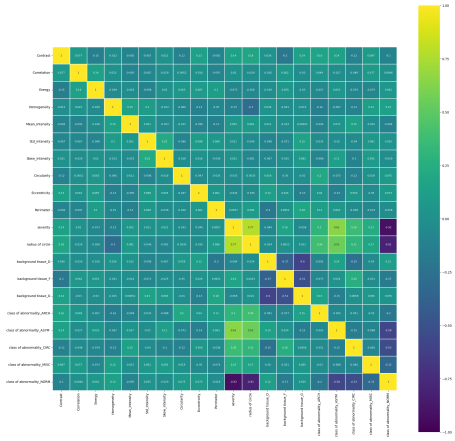


Figure 1: HeatMap Plot

I observed that 19 numerical features exhibited right-skewed distributions. To address this, I applied Box-Cox transformation to normalize their distributions and reduce variance. Following this transformation, I explored two feature selection methods: Sequential Backward Selection (SBS) and Mutual Information (MI). After evaluating both techniques, I decided to proceed with SBS for all models. SBS showed consistently better performance across different classifiers, and given the relatively small number of features in the dataset, it offered a simpler and more interpretable path without the added computational cost and complexity associated with MI.

Because of the imbalanced class distribution mentioned earlier, I used SMOTE (Synthetic Minority Over-sampling Technique) to oversample the training set. This approach helps to mitigate bias toward the majority class and improve generalization. A comparison of class distribution before and after SMOTE is provided in Figure 2. All model training, validation, and testing sets were stratified to preserve class proportions and ensure fair evaluation.

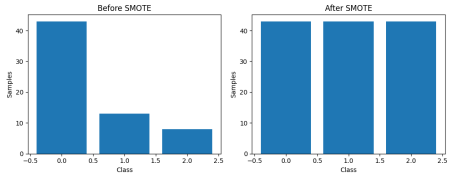


Figure 2: Class Distribution Before and After applying SMOTE.

I also performed t-SNE analysis, as shown in Figure 3, to visualize the impact of SMOTE on class distribution and to justify the models chosen. The results confirmed that SMOTE effectively balances the dataset, revealing more distinct and compact class clusters. This visualization supports the expectation that Random Forest will perform well, as it excels at modeling nonlinear relationships through hierarchical splits and is robust to correlated features due to its use of both bagging and feature bagging. Similarly, Support Vector Machines (SVMs) with nonlinear kernels such as RBF are well-suited for this type of data. They handle high-dimensional spaces effectively and can model complex decision boundaries, with the soft-margin approach helping to mitigate overfitting by allowing some classification errors. Neural Networks are also a promising choice, capable of learning nonlinear patterns and abstract feature representations, particularly valuable for image derived data, though their performance depends heavily on proper tuning and regularization to prevent overfitting. In contrast, Logistic Regression is expected to be the least effective model in this context, as it assumes linear separability, which does not align with the data's nonlinear structure. Nevertheless, it remains a useful baseline model due to its simplicity, interpretability, and support for multiclass classification.

Moreover, I performed hyperparameter tuning using grid search for all models. For Logistic Regression, I tuned the regularization strength parameter C; for Support Vector Machine, I optimized C to control margin softness; for Random Forest, I tuned the number of estimators, maximum depth, and minimum samples to split; and for the Neural Network, I adjusted the hidden layer sizes and learning rate initialization to improve learning performance.

Additionally, to evaluate model performance, I used accuracy, training/validation/test error, 5-fold cross-validation, and weighted

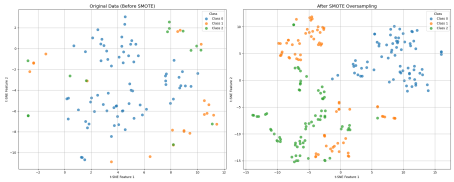


Figure 3: T-sne plot before and after applying SMOTE.

F1 scores. F1 score was particularly important due to the dataset's class imbalance, as it balances precision and recall, offering a better measure of performance on minority classes.

4 RESULTS

I began with logistic regression, which served as a baseline model due to its interpretability and linear nature. After performing SBS-based feature selection, the best-ranked features were: Eccentricity, radius of circle, background tissue D, class of abnormality CIRC, Homogeneity,background tissue F, Skew Intensity, class of abnormality ASYM, Contrast, Std Intensity. You can see in Figure 4 showing logistic regression coefficients visually supports these rankings and illustrates how each feature contributes to the model's multiclass predictions.

Coefficients were interpreted by their sign and magnitude: positive values indicate a higher likelihood of a class as the feature increases, while negative values suggest the opposite. For example, eccentricity showed a strong positive coefficient for Class 2 (malignant), but large negative values for Class 0 and Class 1, suggesting it's highly indicative of malignancy. Radius of circle was also positively associated with malignancy and negatively with normal cases, reinforcing its diagnostic value. Homogeneity was linked more with benign cases than malignant ones, and skew intensity aligned positively with malignancy.

Categorical features also provided insight: background tissue D and background tissue F influenced predictions differently across classes, showing tissue type's impact on classification. Additionally, class of abnormality CIRC suggested circular abnormalities are more benign than malignant. Overall, logistic regression allowed interpretable conclusions about which features drive the predictions for each severity class.

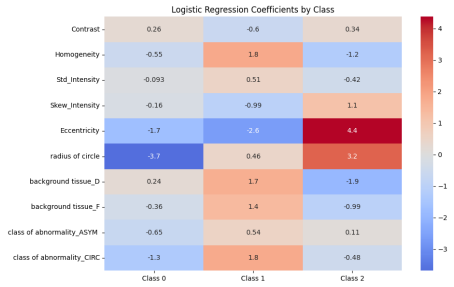


Figure 4: Coefficients of Logistic Regression.

For the Support Vector Machine (SVM) model, the best-performing features were selected using Sequential Backward Selection (SBS), as shown in Figure 5. After hyperparameter tuning, the optimal value of the regularization parameter was found to be C=10, which balances the trade-off between margin maximization and classification error. The most informative features chosen include Contrast, Std Intensity, Skew Intensity, Eccentricity, radius of circle, background tissue D, background tissue F, class of abnormality ASYM, class of abnormality CIRC, and class of abnormality NORM. These features allow the SVM, particularly with a nonlinear kernel, to define flexible decision boundaries that accommodate the complex structure of the data after SMOTE balancing, leading to more accurate classification across severity classes.

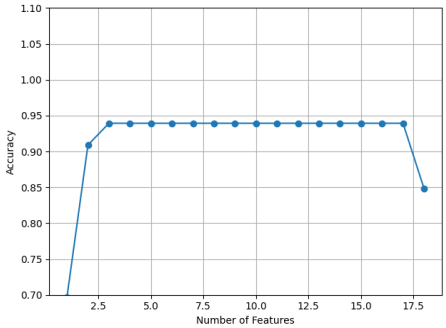


Figure 5: Feature Selection of Support Vector Machine.

For the Random Forest model, feature selection was based on the model's internal feature importance scores rather than Sequential Backward Selection (SBS), as depicted in Figure 6. The most influential features according to the impurity-based ranking included radius of circle, class of abnormality CIRC, class of abnormality ASYM, Skew Intensity, Eccentricity, Mean Intensity, Correlation, Homogeneity, Circularity, and Std Intensity. These features contributed most significantly to the decision-making process of the ensemble trees. After conducting hyperparameter tuning, the optimal parameters were determined to be number of estimators=100, max depth=None, and min samples of split=5. This configuration enabled the model to achieve a good balance between bias and variance while effectively capturing complex nonlinear relationships present in the image-derived features.

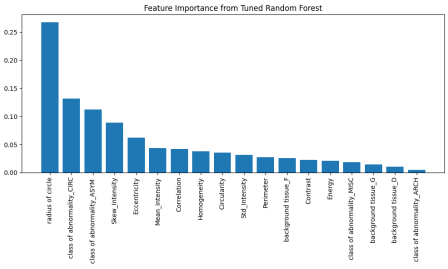


Figure 6: Feature Selection from Random Forests.

For the Neural Network model, hyperparameter tuning identified the optimal configuration as hidden layer sizes = (100,) and learning rate = 0.1. As shown in Figure 7, the loss function decreases

Marlene Fuhr
Rice University
mf88@rice.edu

steadily with each iteration, indicating successful convergence of the training process. Additionally, Figure 8 displays the final decision boundary learned by the neural network, illustrating how the model, with a single hidden layer of 100 neurons and the specified learning rate, is capable of capturing complex, nonlinear separations in the feature space. This supports the model's capacity to generalize well on the classification task after appropriate tuning.

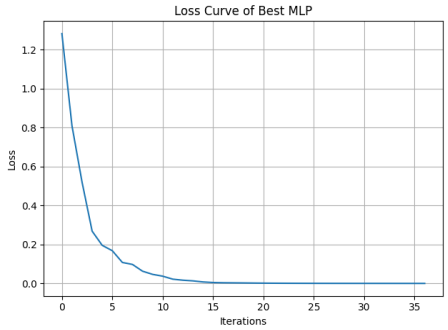


Figure 7: Loss Function of Neural Networks

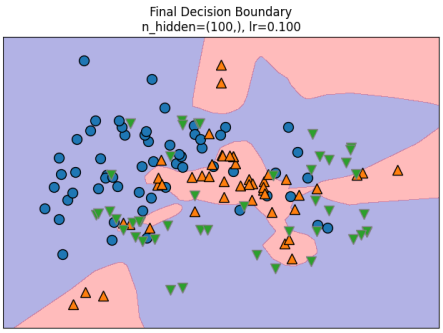


Figure 8: Decision Boundary of Neural Networks

The results can be seen in tables 1, 2,3 and 4.

Model	Train Accuracy	Val Accuracy	Test Accuracy
Logistic Regression	0.9535	0.7273	0.8636
SVM	1.0	0.7273	0.8182
Random Forest	1.0	0.9091	0.8636
Neural Network	1.0	1.0	0.9091

Figure 9: Accuracy Scores

Model	Train Error	Val Error	Test Error
Logistic Regression	0.0465	0.2727	0.1364
SVM	0.0	0.2727	0.1818
Random Forest	0.0	0.0909	0.1364
Neural Network	0.0	0.0	0.0909

Figure 10: Error Rates

Model	F1 Train	F1 Val	F1 Test
Logistic Regression	0.9535	0.7159	0.8636
SVM	1.0	0.6847	0.7756
Random Forest	1.0	0.8955	0.8507
Neural Network	1.0	1.0	0.9091

Figure 11: F-1 Scores

Model	CV Mean Acc	CV Std Dev
Logistic Regression	0.9382	0.0305
SVM	0.976	0.048
Random Forest	0.9603	0.062
Neural Network	0.92	0.0542

Figure 12: Cross Validation Accuracy and Standard Desviation

5 DISCUSSIONS

Based on the results obtained from all models, the Neural Network and Support Vector Machine appear to be the top performing classifiers. The Neural Network achieved perfect training and validation scores and a high test accuracy (0.9091) and F1 score (0.9091), indicating it was able to generalize well while capturing nonlinear patterns in the data. However, its perfect training score also suggests a potential risk of overfitting that should be monitored. The Support Vector Machine also showed strong performance with a test accuracy of 0.8182 and a robust mean cross-validation accuracy of 0.9760, highlighting its stability across folds. Random Forest followed closely, with slightly lower validation and test accuracy than Neural Networks, but excellent generalization due to its ensemble approach. Logistic Regression, while still useful as a baseline, showed the weakest performance overall, particularly in handling the nonlinear separability of the data. Its validation F1 score (0.7159) was notably lower, confirming that linear models may not be sufficient for this classification task. Overall, Neural Networks and SVMs are better suited for this problem due to the dataset's complexity and class overlap.

The Neural Network achieved the highest overall performance, likely due to its capacity to model complex, nonlinear relationships within the extracted features, making it well-suited for the nuanced patterns present in medical imaging data. Random Forest also delivered competitive results, benefiting from its ensemble-based approach that effectively captures feature interactions and reduces overfitting through bagging. While Logistic Regression was the least accurate, it served as a reliable baseline and performed reasonably well on linearly separable components of the dataset. These outcomes highlight the importance of model complexity in addressing nonlinear class boundaries and imbalanced datasets in medical diagnostics.

6 CONCLUSION

In this project, we explored the use of machine learning models to classify breast cancer severity using features extracted from mammographic images in the MIAS dataset. We preprocessed the images with denoising, CLAHE enhancement, and pectoral muscle segmentation, followed by feature extraction, label encoding, and data transformation. Due to the imbalance in the dataset, we applied SMOTE to oversample the minority classes, which significantly improved class distribution as shown through t-SNE visualization.

We evaluated four models: Logistic Regression, Support Vector Machine, Random Forest, and Neural Network, using accuracy, F1

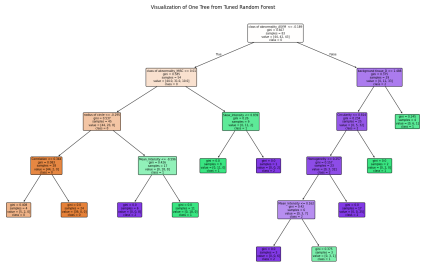


Figure 14: 5 Decision trees for Random Forests

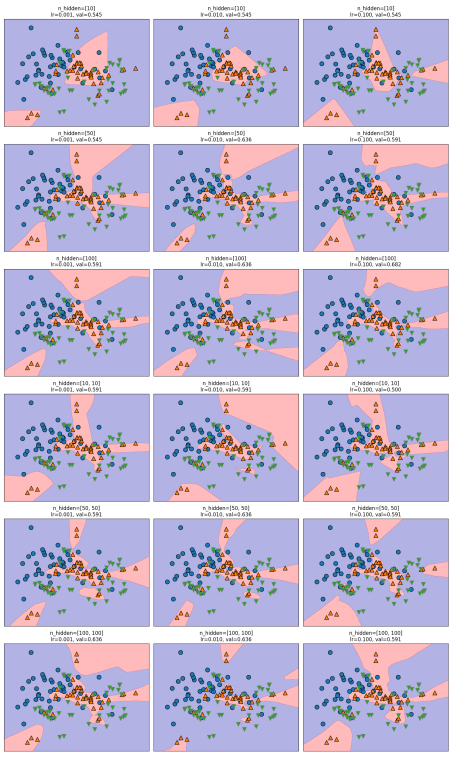
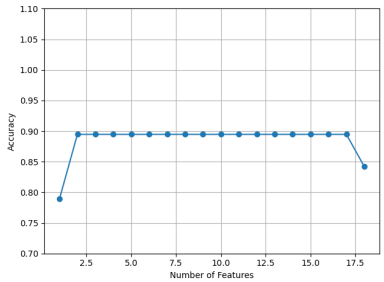
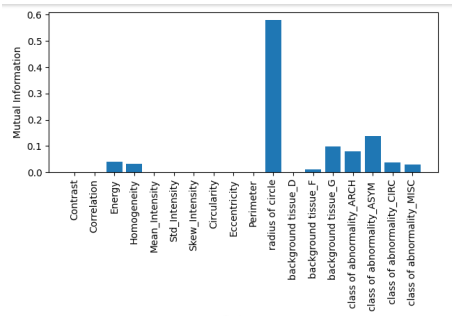


Figure 15: 5 Decision Boundary for Neural Networks



```
Best features: Index(['Contrast', 'Correlation', 'Energy', 'Homogeneity', 'Mean_Intensity',
       'Std_Intensity', 'Circularity', 'class of abnormality_ASYM ',
       'class of abnormality_CIRC', 'class of abnormality_MISC'],
      dtype='object')
Training Accuracy with selected features: 0.8933
Test Accuracy with selected features: 0.9091
```

Figure 16: Feature selection and Perfomance for SBS



```
Training Accuracy with selected features: 0.8933
Test Accuracy with selected features: 0.8788

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        22
           1       0.71      0.71      0.71         7
           2       0.50      0.50      0.50         4

    accuracy                           0.88        33
   macro avg       0.74      0.74      0.74        33
weighted avg       0.88      0.88      0.88        33
```

Figure 17: Feature Selection and Performance for Mutual Information

score, training/validation/test errors, and cross-validation. Among them, the Neural Network performed best, showing strong generalization and accuracy on unseen data. Random Forest also showed excellent results due to its robustness and ability to handle feature interactions. Logistic Regression, while the simplest model, served as an effective baseline.

Overall, our findings suggest that with proper preprocessing, feature selection, and model tuning, machine learning techniques can be powerful tools in supporting breast cancer diagnosis from mammographic data. This work demonstrates a full pipeline, from image to prediction, that can be understood and applied in other medical diagnostic tasks.

REFERENCES

[1] Sharma, S., Mehta, K., & Dodia, Y. (2020). Preprocessing of Breast Cancer Images to Create Datasets for Deep-CNN. *International Journal of Computer Applications*, 975, 8887.

[2] Mekha, G., & Vijayarani, S. (2020). A Review of Breast Cancer Detection Using Machine Learning Techniques. In *Artificial Intelligence and Evolutionary Computations in Engineering Systems* (pp. 111–120). Springer.

[3] Ahmad, M. A., Ali, R., & Iqbal, J. (2020). Intelligent breast cancer prediction empowered with ensemble machine learning. *BMC Bioinformatics*, 21(1), 1-16.

[4] Gupta, D., Khare, S., & Aggarwal, T. (2020). A hybrid deep learning approach for breast cancer detection and classification. *Journal of Ambient Intelligence and Humanized Computing*, 11, 6255–6269.

[5] Abdar, M., et al. (2020). A new machine learning method for detecting breast cancer: Support vector machine with histogram of oriented gradients. In *Intelligent Systems Design and Applications* (pp. 727–737). Springer.

[6] Hiranuma, N., et al. (2019). Learning Interpretable Models with Causal Guarantees. arXiv preprint `arXiv:1905.04247`.

[7] World Health Organization. (2024). Breast cancer. Retrieved from https://www.who.int/news-room/fact-sheets/detail/breast-cancer

7 SOFTWARE

For more information you can access the link in github under the repository named DSCI-303: https://github.com/marlefuhr/DSCI-303

Marlene Fuhr
Rice University
mf88@rice.edu

8 APPENDIX A

In Figure 13 and Figure 14 you can see the whole decision tree and boundary of the Random Forests algorithm for 5 decision. trees.

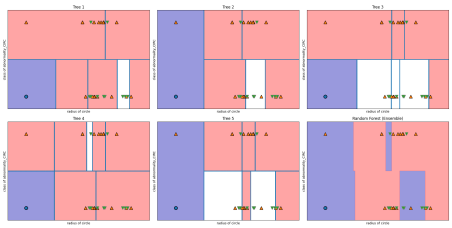


Figure 13: Decision trees for Random Forests

In figure 15 you can see the iterations and decision boundary for Neural Network.

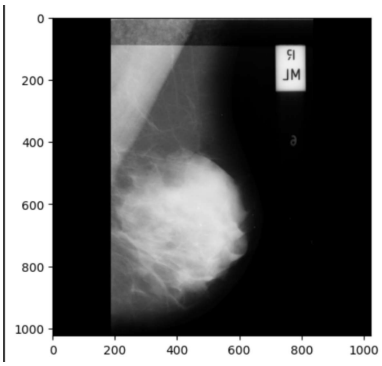
In figure 17 and 16 you can see the feature selection and its performance for SBS and Mutual Information respectfully. I decided to use SBS for support vector machine logistic regression and Neural Networks.

9 APPENDIX B: IMAGE PROCESSING

This Appendix is meant for reading more about the image processing techniques used to extract the features and apply Machine Learning algorithms. This can be found in the follwoing pages.

```
        carpetas.append(carpeta)

plt.imshow(carpetas[1], cmap="gray")
```

2.2. Preprocesamiento



Como podemos ver, algunas imágenes están orientadas a la derecha y otras a la izquierda. Primero, haremos que todas las imágenes estén orientadas a la derecha.

Voltear todas las imágenes hacia la derecha.

```
def right_orient_mammogram(image):
    left_nonzero = cv2.countNonZero(image[:, 0:int(image.shape[1]/2)])
    right_nonzero = cv2.countNonZero(image[:, int(image.shape[1]/2):])

    if(left_nonzero < right_nonzero):
        image = cv2.flip(image, 1)

    return image

flippedImg = np.zeros((120,1024,1024))

for i in range(0,120):
    flippedImg[i] = right_orient_mammogram(cv2.cvtColor(carpetas[i], cv2.COLOR_BGR2GRAY))
```

Volteo manualmente:

Pero algunas imágenes aún no se han volteado, por lo que lo estamos haciendo manualmente. (Podemos mejorar nuestro algoritmo para eliminar este volteo manual).

```
def forceFlip(img):
    img = cv2.flip(img,1)
    return img

flippedImg[35]=forceFlip(flippedImg[35])
flippedImg[75]=forceFlip(flippedImg[75])
flippedImg[101]=forceFlip(flippedImg[101])
```

Para ver las 120 imágenes volteadas dirigirse a la sección 2 del anexo.

Recorte del lado izquierdo de la imagen Recortaremos el lado izquierdo de las imágenes. Todas las imágenes deberían tener un tamaño de 1024x800 después de este paso.

Algoritmo para cropLeft()

Recortar el lado izquierdo. Si el ancho de la imagen resultante es igual a 800, devolver la imagen. Si el tamaño de la imagen resultante es menor que 800, añadir píxeles en el lado derecho para hacerla de 800 píxeles de ancho. Si el ancho de la imagen resultante es mayor a 800, recortar los píxeles del lado derecho para ajustarla a 800 píxeles de ancho.

```
def cropLeft(img):
    mini = 0
```

```
    pix = 0

    for i in range(1024):
        if img[10][i]>10:
            mini = i
            break
    newimg=np.ones((1024,800))*pix
#     print(min,img[10][200])
    diff = 1024-mini
    if diff<800:
        newimg[:,:diff] = img[:,mini:]
#         print('in cropleft : shape :', newimg.shape)
    elif diff>800:
        newimg = img[:,mini:(mini+800)]
    elif diff==800:
        newimg = img[:,mini:]
#     print(newimg.shape)

#     plt.imshow(newimg)
    return newimg

leftCropped = np.zeros((120,1024,800))
for i in range(0,120):
    imgforcropleft = flippedImg[i]
    leftCropped[i] = cropLeft(imgforcropleft)
```

Sacar artefactos:

Artefactos son las etiquetas en las mamografías (Los artefactos pueden observarse en la imagen de referencia anterior). Estos artefactos pueden generar resultados negativos, por lo que es importante eliminarlos.

Algoritmo para removeArt()

Identificar el inicio de la parte brillante y almacenarlo en la variable start. Identificar el final de la parte brillante y almacenarlo en la variable end. Establecer todos los píxeles a la derecha de end en cero. Devolver la imagen.

```
def removeArt(img):
    thresh = 30
#     thresh = 10
    minPix = 40


#     print(start)

    newim = np.zeros((1024,800))
    newim = img.copy()
    for i in range(1024):
        start = 0
        for k in range(800):
            if img[100][k] != 0:
                start = k
                break
        for j in range(start , 800-minPix):
#         for j in range(768-minPix):
            ar = newim[i][j:j+minPix]
            if (all(x < thresh for x in ar)):
                newim[i][j:] = 0
                break
#     plt.imshow(newim)
    return newim

artImg = np.zeros((120,1024,800))
for i in range(0,120):
    artImg[i]  = removeArt(leftCropped[i])
```

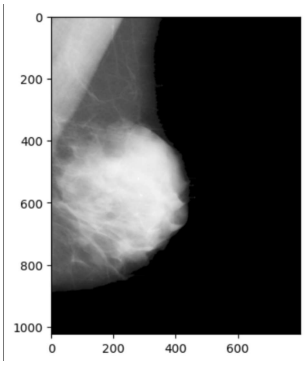
Recortando la parte superior Algunas imágenes tienen todos los valores en cero en las filas superiores, por lo que las recortaremos.

```
def cropTop(img):
    newim = np.zeros((1024,800))
```

```
    newim[:1023,:]=img[1:,:]
    return newim

topCropped = np.zeros((120,1024,800))
for i in range(0,120):
    topCropped[i]  = cropTop(artImg[i])
```

Recorte individual de algunas imágenes Ahora, algunas imágenes tienen más filas con valores cero en la parte superior, por lo que se eliminan manualmente.

```
new1 = np.zeros((1024,800))
new1[:934,:] = topCropped[1][90:,:]
topCropped[1] = new1

new1 = np.zeros((1024,800))
new1[:1002,:] = topCropped[10][22:,:]
topCropped[10] = new1

for i in range(0,120):
  topCropped[i]= cv2.normalize(
        topCropped[i],
        None,
        alpha=0,
        beta=255,
        norm_type=cv2.NORM_MINMAX,
        dtype=cv2.CV_32F,
    )
  topCropped[i]= topCropped[i].astype("uint8")

def robust_convert_to_uint8(image):
    """
    Convierte cualquier matriz de imagen a tipo uint8, asegurando que los valores est n en el rango [0, 2

    Args:
        image (np.ndarray): Imagen de entrada.

    Returns:
        np.ndarray: Imagen convertida a uint8.
    """
    # Verificar que sea una matriz NumPy
    if not isinstance(image, np.ndarray):
        raise TypeError("La␣entrada␣debe␣ser␣una␣matriz␣NumPy.")

    # Verificar que la matriz no est  vac a
    if image.size == 0:
        raise ValueError("La␣imagen␣de␣entrada␣est ␣vac a.")

    # Normalizar si los valores son mayores que 255 o menores que 0
    if image.dtype != np.uint8:
        min_val, max_val = image.min(), image.max()
        print(f"Valores␣m nimos␣y␣m ximos␣antes␣de␣normalizar:␣{min_val},␣{max_val}")

        if min_val < 0 or max_val > 255:
            # Normalizar al rango [0, 255]
            image = (image - min_val) / (max_val - min_val) * 255.0
            print("La␣imagen␣fue␣normalizada␣al␣rango␣[0,␣255].")

        # Convertir a tipo uint8
        image = np.clip(image, 0, 255).astype(np.uint8)
        print("Imagen␣convertida␣a␣uint8.")

    return image

carpetas1= np.zeros((120,1024,800))
for i in range(0,120):
    carpetas1[i]  = cropTop(artImg[i])

for i in range(0,120):
  carpetas1[i] = robust_convert_to_uint8(topCropped[i])
```



Para ver las 120 imágenes con los artefactos removidos en la sección 3 del anexo.

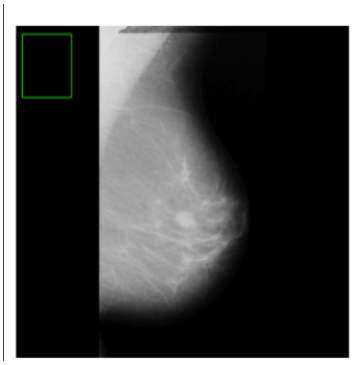
2.3. Estimar el ruido

Con la siguiente función de rectángulo, analizamos una región de interés ROI homogenea seleccionada en cada una de las imágenes de la carpeta.

```
def get_rectangle_intensities(image, rect_start, B, H):
    rect_end = (rect_start[0] + B, rect_start[1] + H) #esquina inferior
    rect_intensities = image[rect_start[1]:rect_end[1], rect_start[0]:rect_end[0]]

    if image.dtype != np.uint8:
        image = cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)


    if len(image.shape) == 2:  # Escala de grises
        image_with_rect = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
    else:
        image_with_rect = image.copy()

    cv2.rectangle(image_with_rect, rect_start, rect_end, (0, 255, 0), 2)
    return image_with_rect, rect_intensities
```

A continuación, elegir un vertice del rectangulo, la base y la altura. Aplicando las funciones realizadas anteriormente, obtenemos la matriz de intensidades de píxeles dentro del rectangulo elegido, el cual es posicionado en el fondo negro de la imágen donde no se encuentra la mama.

```
image_with_rect = [np.empty([1024,800])]
for i in range(0,120):
    image_with_rect.append(np.empty([1024,800]))

rect_intensities = [np.empty([1024,800])]
for i in range(0,120):
    rect_intensities.append(np.empty([1024,800]))

for i in range(0,120):
  rect_start = (20, 25)
  B, H = 150, 195
  image_with_rect[i], rect_intensities[i] = get_rectangle_intensities(carpetas[i], rect_start, B, H)
  #print("Imagen:",i+1,"Matriz de intensidades del rect ngulo:\n", rect_intensities)



plt.figure(figsize=(10,520))
for i in range(0,120):
    plt.subplot(120,1, i+1)
    plt.imshow(image_with_rect[i], cmap="gray")
    plt.axis('off')

plt.show()
```



Podemos calcular las estadisticas dentro de una region de interés o dentro de toda la imagen. En este caso, voy a calcular las estadisticas de las regiones de interes.A continuación, estimamos el ruido basado en el histograma y calculando la varianza en la región de interés homogenea.

```
def calculate_statistics(image):

  hist = cv2.calcHist([image], [0], None, [256], [0, 256])
  mean_value = np.mean(image)
  variance_value = np.var(image)
  return hist, mean_value, variance_value

hist_rect = [np.empty([195,150])]
for i in range(0,120):
    hist_rect.append(np.empty([195,150]))
mean_rect = [np.empty([195,150])]
for i in range(0,120):
    mean_rect.append(np.empty([195,150]))
var_rect = [np.empty([195,150])]
for i in range(0,120):
    var_rect.append(np.empty([195,150]))
for i in range(0,120):
  hist_rect[i], mean_rect[i], var_rect[i] = calculate_statistics(rect_intensities[i])

for i in range(0,120):
  print("Imagen:",i+1,"varianza:",var_rect[i],"media:",mean_rect[i])
```

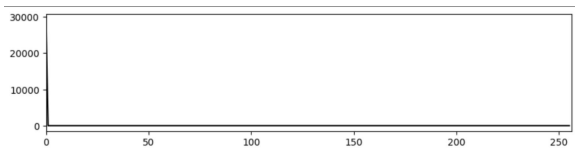
1. Imagen: 1, varianza: 0.0, media: 0.0
2. Imagen: 2, varianza: 0.0, media: 0.0

   .....

3. Imagen: 119, varianza: 0.0, media: 0.0
4. Imagen: 120, varianza: 0.0, media: 0.0

Para ver la lista completa de las estadísticas de las 120 imágenes en la sección 4 del anexo.

```
plt.figure(figsize=(10,320))
for i in range(0,120):
    plt.subplot(120,1, i+1)
    plt.plot(hist_rect[i], color='black')
    plt.xlim([0, 256])

plt.show()
```




```
def histogram_noise_analysis(roi):
    hist = cv2.calcHist([roi], [0], None, [256], [0, 256])
```

```
    hist = hist / hist.sum()
    mean = np.mean(roi)
    std_dev = np.std(roi)
    dma = np.mean(np.abs(roi - mean))
    # Calcular la frecuencia del valor de intensidad m s frecuente (modo)
    mode_val = np.argmax(hist)
    stats = {
        'Media (ROI)': mean,
        'Desviaci n Est ndar (  )': std_dev,
        'Desviaci n Media Absoluta (DMA)': dma,
        'Modo del Histograma': mode_val
    }
    return stats, hist

plt.figure(figsize=(10,320))
for i in range(0,120):
  stats, hist=histogram_noise_analysis(rect_intensities[i])
  print("Estad sticas adicionales de la imagen n mero:",i+1)
  for key, value in stats.items():
    print(f"{key}: {value}")
```

Para ver los histogramas completos de las 120 imágenes ir a la sección 5 del anexo.

1. Estadísticas adicionales de la imagen número: 1
    - Media (ROI): 0.0
    - Desviación Estándar (σ): 0.0
    - Desviación Media Absoluta (DMA): 0.0
    - Modo del Histograma: 0

Estadísticas adicionales de las 120 imágenes se encuentran en la sección 4 del anexo. Como se puede ver las imágenes no parecen tener ruido relevante en las imágenes.

A continuación, estimamos el ruido a partir del espectro de potencia basado en la transformada de fourier. El espectro de potencia puede revelar la presencia de componentes de alta frecuencia que corresponderían al ruido.

Estimar el ruido con la transformada de fourier con la imagen completa:

```
def fourier_transform(image):
    f = np.fft.fft2(image)
    fshift = np.fft.fftshift(f)
    power_spectrum = np.abs(fshift) ** 2
    return power_spectrum, fshift

def estimate_noise_fourier(image):
    f = np.fft.fft2(image)
    fshift = np.fft.fftshift(f)
    magnitude_spectrum = 20 * np.log(np.abs(fshift))
    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2
    mask = np.ones((rows, cols), np.uint8)
    r = 30
    center_circle = np.ogrid[:rows, :cols]
    mask_area = (center_circle[0] - crow) ** 2 + (center_circle[1] - ccol) ** 2 <= r*r
    mask[mask_area] = 0
    high_freq_magnitudes = magnitude_spectrum[mask == 1]
    noise_estimate = np.mean(high_freq_magnitudes)
    return magnitude_spectrum, noise_estimate

for i in range(0,120):
  #magnitude_spectrum, noise_estimate = estimate_noise_fourier(cv2.cvtColor(carpetas[i].copy(), cv2.COLOR_
  magnitude_spectrum, noise_estimate = estimate_noise_fourier(topCropped[i])
  plt.figure(figsize=(30, 420))
  plt.subplot(120, 1, 1)
  plt.title('Espectro de Fourier')
  plt.imshow(magnitude_spectrum, cmap='gray')
  plt.axis('off')
  plt.subplot(120, 2, 2)
```

```
  plt.text(0.5, 0.5, f'Ruido Estimado: {noise_estimate:.0f}', fontsize=12, ha='center')
  plt.axis('off')
plt.show()
```

Las estimaciones de ruido para las 120 imágenes se pueden visaulizarse en la sección 6 del anexo.

```
pip install PyWavelets

import pywt
def wavelet_noise_estimation(image, wavelet='haar', level=1):
    image = image.astype(np.float32)
    # Aplicar la transformada wavelet discreta
    coeffs = pywt.wavedec2(image, wavelet=wavelet, level=level)
    LH, HL, HH = coeffs[1]
    sigma_LH = np.std(LH)
    sigma_HL = np.std(HL)
    sigma_HH = np.std(HH)
    noise_estimation = np.mean([sigma_LH, sigma_HL, sigma_HH])

    reconstructed_image = pywt.waverec2(coeffs, wavelet)
    reconstructed_image = np.clip(reconstructed_image, 0, 255).astype(np.uint8)

    return noise_estimation, reconstructed_image

for i in range(0,120):
  noise_level, transformed_image = wavelet_noise_estimation(carpetas1[i], wavelet='haar', level=1)
  print(f"Imagen n mero :{i+1}\nEstimaci n del nivel de ruido: {noise_level}")
```

1. Imagen número: 1
   Estimación del nivel de ruido: 1.8689807653427124
2. Imagen número: 2
   Estimación del nivel de ruido: 1.802535057067871

   .....
3. Imagen número: 119
   Estimación del nivel de ruido: 2.1040849685668945
4. Imagen número: 120
   Estimación del nivel de ruido: 2.068614959716797

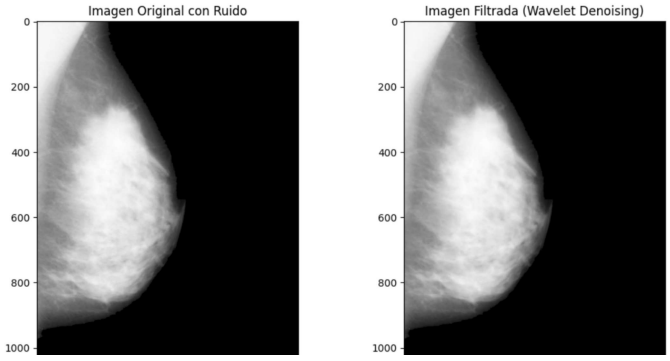
La tabla completa se puede ver en la sección 7 del anexo.

2.4. Denoising

Transformada Wavelet

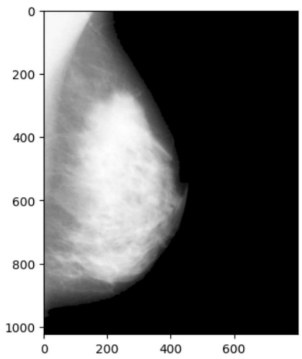
```
# Par metros del ruido ya calculados de la imagen
varianza_ruido = 1  # ejemplo de varianza conocida
#img=cv2.cvtColor(carpetas[2].copy(), cv2.COLOR_RGB2GRAY)
img=carpetas1[2]
# Calcular sigma a partir de la varianza existente del ruido
```

```
sigma = np.sqrt(varianza_ruido)

# Realizar la transformada wavelet discreta (DWT)
coeffs = pywt.wavedec2(img, 'haar', level=2)

# Extraer coeficientes
LL2, (LH2, HL2, HH2), (LH1, HL1, HH1) = coeffs

# Calcular el umbral utilizando la varianza existente del ruido
threshold = sigma * np.sqrt(2 * np.log2(img.size))
print(f"Threshold calculado: {threshold}")

# Umbralizar cada sub-banda utilizando soft thresholding
def soft_thresholding(coeffs, threshold):
    return pywt.threshold(coeffs, threshold, mode='soft')

LL2_thresh = soft_thresholding(LL2, threshold)
LH2_thresh = soft_thresholding(LH2, threshold)
HL2_thresh = soft_thresholding(HL2, threshold)
HH2_thresh = soft_thresholding(HH2, threshold)

# Reconstruir la imagen a partir de los coeficientes umbralizados
coeffs_thresh = [LL2_thresh, (LH2_thresh, HL2_thresh, HH2_thresh), (LH1, HL1, HH1)]
denoised_img = pywt.waverec2(coeffs_thresh, 'haar')
denoised_img = np.clip(denoised_img, 0, 255)

# Mostrar resultados
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title('Imagen Original con Ruido')
plt.imshow(img, cmap='gray')

plt.subplot(1, 2, 2)
plt.title('Imagen Filtrada (Wavelet Denoising)')
plt.imshow(denoised_img, cmap='gray')
plt.show()
transformed_image1=denoised_img
```



Filtro promediador

```
promediador1 = cv2.blur(carpetas1[2], (3, 3))  # Filtro promediador con kernel de 3x3
plt.imshow(promediador1, cmap='gray')
```




```
def calculate_snr(image, noise):
    signal_power = np.mean(image**2)
    noise_power = np.mean(noise**2)
    snr = abs(10 * np.log10(signal_power / noise_power))
    return snr

def calculate_psnr(original, noisy):
    mse = np.mean((original - noisy) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    psnr = 10 * np.log10((max_pixel ** 2) / mse)
    return psnr

snr_promediador = calculate_snr(topCropped[2],promediador1)
psnr_promediador = calculate_psnr(topCropped[2],promediador1)
print("Imagen␣:")
print(f"PSNR␣(promediador):␣{psnr_promediador}␣dB")
print(f"SNR␣(promediador):␣{snr_promediador}␣dB")

wavelet1_r=transformed_image1
snr_wavelet = calculate_snr(topCropped[2],wavelet1_r)
psnr_wavelet = calculate_psnr(topCropped[2],wavelet1_r)
print("Imagen:")
print(f"PSNR␣(wavelet):␣{psnr_wavelet}␣dB")
print(f"SNR␣(wavelet):␣{snr_wavelet}␣dB")
```

1. Imagen:
   PSNR (promediador): 43.6839861797788 dB
   SNR (promediador): 0.00049508421146909 dB
   PSNR (wavelet): 45.08653234473027 dB
   SNR (wavelet): 0.07186126112193565 dB

Decidimos usar filtro promediador aunque los resultados sean muy semejantes. Laas 120 imágenes con el filtro promediador se encuentran en la sección 8 en el anexo.

```
promediador = []  # Crear una lista para almacenar los resultados

for i in range(0, 120):
    promediadores = cv2.blur(carpetas1[i], (3, 3))
    promediador.append(promediadores)
```

Utilizamos la técnica de CLAHE para mejorar el contrastre

```
def clahe(img, clip=2.0, tile=(8, 8)):
    img = cv2.normalize(
        img,
        None,
        alpha=0,
        beta=255,
        norm_type=cv2.NORM_MINMAX,
        dtype=cv2.CV_32F,
    )
    img_uint8 = img.astype("uint8")
```

```
    clahe_create = cv2.createCLAHE(clipLimit=clip, tileGridSize=tile)
    clahe_img = clahe_create.apply(img_uint8)

    return clahe_img

enhancedImg = np.zeros((120,1024,800))

for i in range(0,120):
    enhancedImg[i] = clahe(promediador[i])

plt.imshow(enhancedImg[0], cmap='gray')
```

Las 120 imágenes con CLAHE se encuentran en la sección 9 del anexo para visualizar libremente.

2.5. Sacar el músculo pectoral: Canny edge y transformada Hough

Eliminar el pectoral Esta parte es muy crucial.

Los músculos pectorales aparecen en las esquinas superiores izquierdas de las mamografías. Estos músculos tienen la misma densidad que los tejidos cancerosos, por lo que es necesario eliminarlos. Dado que estos músculos tienen la misma densidad que las células cancerígenas, resulta un poco complicado eliminarlos. Cada paso se realiza como una función separada. Los algoritmos de cada paso se encuentran junto con sus respectivas funciones.

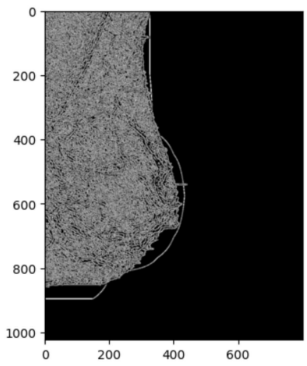
```
merg = np.zeros((120,1024,800))

i = 0
start = time.time()
for image in enhancedImg:
    # Verifica si la imagen no est  en uint8 y convi rtela
    if image.dtype != np.uint8:
        image = (image * 255).astype(np.uint8) if image.max() <= 1 else image.astype(np.uint8)

    # Crear kernel
    kernel = np.ones((120, 120), np.uint8)

    # Operaciones morfol gicas
    erosion = cv2.erode(image, kernel, iterations=1)
    dilation = cv2.dilate(erosion, kernel, iterations=1)

    # Aseg rate de que la m scara sea uint8
    if dilation.dtype != np.uint8:
        dilation = dilation.astype(np.uint8)

    # Aplicar bitwise_and con la m scara
    merged = cv2.bitwise_and(image, image, mask=dilation)
    merg[i] = merged

    i += 1

end = time.time()
print(f"Tiempo␣total:␣{end␣-␣start}␣segundos")
```

Operaciones morfológicas Tiempo total: 2.1897337436676025 segundos

Transformada Hough y Canny Edges

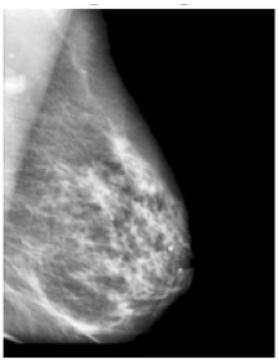
```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import pylab as pylab
from skimage import io
from skimage import color


from skimage.feature import canny
from skimage.filters import sobel
def apply_canny(image):
    sigma = 0.5  # Ajusta el sigma para suavizar  m  s/menos
    canny_img = canny(image, sigma=sigma)
    return sobel(canny_img)

from skimage.transform import hough_line, hough_line_peaks

def get_hough_lines(canny_img):
    h, theta, d = hough_line(canny_img)
    lines = []
    print('\nAll hough lines')
    for _, angle, dist in zip(*hough_line_peaks(h, theta, d)):
        if np.sin(angle) == 0:  # Evitar divisi n por cero
            print(f"Skipping line with angle {np.degrees(angle):.2f}  (sin(angle) = 0)")
            continue
        print(f"Angle: {np.degrees(angle):.2f}, Dist: {dist:.2f}")

        x1 = 0
        y1 = int((dist - x1 * np.cos(angle)) / np.sin(angle))
        x2 = canny_img.shape[1]
        y2 = (dist - x2 * np.cos(angle)) / np.sin(angle)
        lines.append({
            'dist': float(dist),
            'angle': np.degrees(float(angle)),
            'point1': [x1, y1],
            'point2': [x2, y2]
        })
    return lines

def shortlist_lines(lines):
    MIN_ANGLE = 10
    MAX_ANGLE = 70
    MIN_DIST = 5
    MAX_DIST = 256
    print(f"Total detected lines: {len(lines)}")
    shortlisted_lines = [
        x for x in lines if
        (x['dist'] >= MIN_DIST) and
        (x['dist'] <= MAX_DIST) and
        (x['angle'] >= MIN_ANGLE) and
        (x['angle'] <= MAX_ANGLE)
    ]
    print(f"Shortlisted lines: {len(shortlisted_lines)}")
    for i in shortlisted_lines:
        print(f"Angle: {i['angle']:.2f}, Dist: {i['dist']:.2f}")
    return shortlisted_lines

from skimage.draw import polygon
def remove_pectoral(shortlisted_lines):
    if not shortlisted_lines:
        print("No se encontraron l neas para eliminar el m sculo pectoral.")
        return [], []
    shortlisted_lines.sort(key=lambda x: x['dist'])
    pectoral_line = shortlisted_lines[0]
    d = pectoral_line['dist']
    theta = np.radians(pectoral_line['angle'])
    x_intercept = d / np.cos(theta)
    y_intercept = d / np.sin(theta)
```

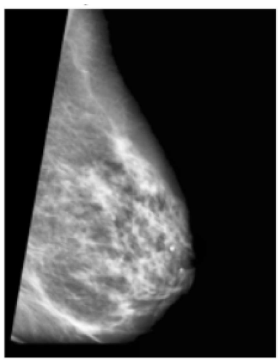
```
    return polygon([0, 0, y_intercept], [0, x_intercept, 0])
def draw_hough_lines(image, lines):
    """
    Dibuja l neas Hough sobre la imagen usando OpenCV.
    """
    # Crea una copia en formato BGR para superponer l neas de colores
    hough_image = cv2.cvtColor((image * 255).astype(np.uint8), cv2.COLOR_GRAY2BGR)

    for line in lines:
        x1, y1 = map(int, line['point1'])
        x2, y2 = map(int, line['point2'])
        cv2.line(hough_image, (x1, y1), (x2, y2), (0, 255, 0), 2)  # Verde
    return hough_image

def display_image(filename):
    image = read_image(filename)
    canny_image = apply_canny(image)
    lines = get_hough_lines(canny_image)
    shortlisted_lines = shortlist_lines(lines)
    rr, cc = remove_pectoral(shortlisted_lines)
    image[rr, cc] = 0
    cv2.imwrite('image13.png', image)

canny_vector = np.zeros((120,1024,800))

for i in range(0,120):
    canny_vector[i] = apply_canny(merg[i])}

canny_image = apply_canny(merg[0])
plt.imshow(canny_image, cmap='gray')
```




```
lines_vector = []
short_vector = []

for i in range(120):
    image = canny_vector[i]
    if image.dtype != np.uint8:
        image = (image * 255).astype(np.uint8) if image.max() <= 1 else image.astype(np.uint8)

    lines = get_hough_lines(image)  # Lista de diccionarios
    shortlisted = shortlist_lines(lines)  # Lista filtrada

    lines_vector.append(lines)  # Almacena la lista completa
    short_vector.append(shortlisted)  # Almacena la lista filtrada

lines = get_hough_lines(canny_image)
shortlisted_lines = shortlist_lines(lines)

# Procesar cada copia y almacenar el resultado
for i in range(120):
    if not short_vector[i]:  # Verificar si no hay l neas
        print(f"No se encontraron l neas para la imagen {i}.")
        remove_vector.append(copia_merg[i])  # Guardar la imagen sin modificar
        continue
```

```
    rr, cc = remove_pectoral(short_vector[i])  # Coordenadas del m sculo pectoral

    # Validar que los  ndices  est n dentro de los l mites de la imagen
    rr = np.clip(rr, 0, copia_merg[i].shape[0] - 1).astype(int)
    cc = np.clip(cc, 0, copia_merg[i].shape[1] - 1).astype(int)

    copia_merg[i][rr, cc] = 0  # Eliminar la regi n del m sculo
    remove_vector.append(copia_merg[i])  # Guardar la imagen procesada

for i in range(0,120):
  plt.figure(figsize=(30, 420))
  plt.subplot(120, 1, 1)
  plt.title('Imagen␣original')
  plt.imshow(merg[i], cmap='gray')
  plt.axis('off')
  plt.subplot(120, 2, 2)
  plt.title(f'M sculo␣pectoral␣sacado␣{i}')
  plt.imshow(remove_vector[i], cmap='gray')
  plt.axis('off')
plt.show()
```





2.6. Extracción de características para Técnicas de Machine Learning

```
import os
import math
import h5py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy as sp
from scipy.stats import skew
from scipy.stats import kurtosis
import scipy.ndimage as ndi
import sklearn.metrics as metrics
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn import svm
from skimage.measure   import shannon_entropy
import cv2
from skimage import io
from skimage import color
from skimage.draw import polygon
#from skimage.feature import greycomatrix, greycoprops
#from skimage.measure import greycomatrix, greycoprops
from sklearn.decomposition import PCA
import pandas as pd
import numpy as np
import h5py
from skimage.feature import canny
from skimage.filters import sobel
```

```
from skimage.transform import hough_line, hough_line_peaks
from sklearn.metrics import classification_report, accuracy_score

with h5py.File('../input/final.h5', 'r') as scan_H5:
    print("Keys: %s" % scan_H5.keys())
    severity =  scan_H5['severity'][:]
    scanH5 = scan_H5['img'][:]
    print('\n|||||||File read successfuly|||||||')
```

Extracción de características: Pasar de categoricas a numerica

```
df = pd.DataFrame({'severity':severity})
df['severity'] = df['severity'].str.decode("utf-8")
df['severity'].replace('B',1,inplace=True)
df['severity'].replace('M',2,inplace=True)
df['severity'].replace('nan',3,inplace=True)
```

Sacar células nromales

```
li = []
for i in range(330):
    if df['severity'][i] != 3:
        li.append(i)

deletedScan = np.take(scanH5, li, axis=0)
df = df[df['severity'] != 3]
Y = df.values
Y = Y.flatten()

finalImage  = deletedScan
```

Características estadísticas: featureVector = pd.DataFrame() Media

```
row=35
means = np.zeros((row,3))
for i in range(row):
    means[i] = cv2.mean(finalImage[i])[:3]

featureVector['means'] = means[:,0]
```

Desviación Estándar

```
stds = np.zeros((row,3))
for i in range(row):
    _,stds[i] = cv2.meanStdDev(finalImage[i])

featureVector['stds'] = stds[:,0]
```

Kurtosis

```
kurtos = np.zeros(row,)

for i in range(row):
    kurtos[i] = np.average(kurtosis(finalImage[i]))

featureVector['kurtosis'] = kurtos
```

Asimetría

```
skewness = np.zeros(row,)

for i in range(row):
    skewness[i] = np.average(skew(finalImage[i]))

featureVector['skewness'] = skewness
```

Suavidad

```
smoothness = np.zeros((row,))

for i in range(row):
    smoothness[i] = np.average(np.absolute(ndi.filters.laplace(finalImage[i]).astype(float) / 255.0))
featureVector['smooth'] = smoothness
```

Entropía

```
entropyFromImage = np.zeros(row,)
for i in range(row):
    entropyFromImage[i] = shannon_entropy(finalImage[i])
featureVector['entropy'] = entropyFromImage
```

Características GLCM

```
entropies = np.zeros((row,))
dissimilarity = np.zeros((row,))
correlation = np.zeros((row,))
homogeneity = np.zeros((row,))
energy = np.zeros((row,))
contrast= np.zeros((row,))
ASM= np.zeros((row,))

def getGlcm():
    for i in range(row):
        glcm = greycomatrix(finalImage[i].astype('uint8'), distances=[1],
                            angles=[0], symmetric=True,
                            normed=True)
#         print(glcm.shape)

        dissimilarity[i]= greycoprops(glcm, 'dissimilarity')[0, 0]
        correlation[i]= greycoprops(glcm, 'correlation')[0, 0]
        homogeneity[i] = greycoprops(glcm,'homogeneity')[0, 0]
        energy[i] = greycoprops(glcm, 'energy')[0, 0]
        contrast[i]= greycoprops(glcm, 'contrast')[0, 0]
        ASM[i] = greycoprops(glcm,'ASM')[0,0]
        glcm1 = np.squeeze(glcm)
        entropies[i] = -np.sum(glcm1*np.log2(glcm1 + (glcm1==0)))
```

```
getGlcm()

featureVector['entropies'] = entropies
featureVector['dissimilarity'] = dissimilarity
featureVector['correlation'] = correlation
featureVector['homogeneity'] =homogeneity
featureVector['energy'] = energy
featureVector['contrast'] = contrast
featureVector['ASM'] =ASM

featureVector.head(5)
```

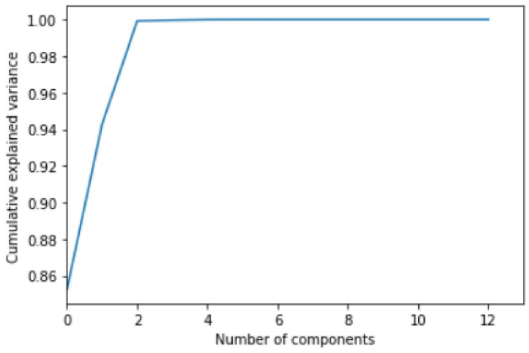
	means	stds	kurtosis	skewness	smooth	entropy	entropies	dissimilarity	correlation	homogeneity	energy	contrast	ASM
0	34.640250	69.033719	-1.315060	0.421231	0.010550	2.368539	3.324533	1.030793	0.997564	0.834578	0.779962	23.242229	0.608341
1	37.305714	72.430764	-1.757026	0.393993	0.010108	2.440434	3.380427	0.950110	0.997965	0.836059	0.772250	21.371413	0.596371
2	73.054004	78.400490	-0.287723	-0.546568	0.025749	4.106596	6.224676	2.319036	0.996399	0.632705	0.523306	44.290387	0.273849
3	73.054004	78.400490	-0.287723	-0.546568	0.025749	4.106596	6.224676	2.319036	0.996399	0.632705	0.523306	44.290387	0.273849
4	47.341917	75.290774	-1.717753	0.011414	0.015706	2.885099	4.210470	1.525751	0.996866	0.770632	0.706395	35.551051	0.498994

Cuadro 1: Estadísticas y características de las imágenes.

Feature selection

```
pca = PCA().fit(featureVector)
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlim(0,13,1)
```

```
plt.xlabel('Number of components')
plt.ylabel('Cumulative explained variance')
```




```
sklearn_pca = PCA(n_components=3)
X = sklearn_pca.fit_transform(featureVector)

# test
X = featureVector
```

Entrenamiento y testeo

```
X_train, X_test = train_test_split(X,
                                   test_size = 0.25,
                                   random_state = 2021)

Y_train,Y_test = train_test_split(Y,test_size = 0.25,random_state = 2021)

print('train', Y_train.shape[0], 'Test', Y_test.shape[0])
```

train 92 Test 31

3. Resultados

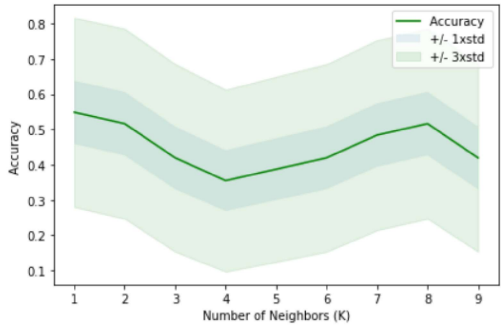
3.1. k-Nearest Neighbors (k-NN)

Creación del modelo

```
Ks = 10
mean_acc = np.zeros((Ks-1))
std_acc = np.zeros((Ks-1))

for n in range(1,Ks):

    #Train Model and Predict
    neigh = KNeighborsClassifier(n_neighbors = n).fit(X_train,Y_train)
    yhat=neigh.predict(X_test)
    mean_acc[n-1] = metrics.accuracy_score(Y_test, yhat)


    std_acc[n-1]=np.std(yhat==Y_test)/np.sqrt(yhat.shape[0])

mean_acc
```

array([0.5483871 , 0.51612903, 0.41935484, 0.35483871, 0.38709677, 0.41935484, 0.48387097, 0.51612903, 0.41935484])

```
plt.plot(range(1,Ks),mean_acc,'g')
plt.fill_between(range(1,Ks),mean_acc - 1 * std_acc,mean_acc + 1 * std_acc, alpha=0.10)
plt.fill_between(range(1,Ks),mean_acc - 3 * std_acc,mean_acc + 3 * std_acc, alpha=0.10,color="green")
plt.legend(('Accuracy ', '+/- 1xstd','+/- 3xstd'))
plt.ylabel('Accuracy ')
plt.xlabel('Number of Neighbors (K)')
plt.tight_layout()
plt.show()
```




```
print( "El mejor accuracy es con", mean_acc.max(), "con k=", mean_acc.argmax()+1)
```

El mejor accuracy es con 0.5483870967741935 con k= 1

```
# should find K
knn = KNeighborsClassifier(1)
knn.fit(X_train, Y_train)
Y_pred_knn = knn.predict(X_test)
print('Accuracy %2.2f%%' % (100*accuracy_score(Y_test, Y_pred_knn)))
print(classification_report(Y_test, Y_pred_knn))
```

Clase	Precision	Recall	F1-Score	Support
1	0.58	0.65	0.61	17
2	0.50	0.43	0.46	14
Accuracy			0.55	31
Macro avg	0.54	0.54	0.54	31
Weighted avg	0.54	0.55	0.54	31

Cuadro 2: Métricas de rendimiento del modelo

3.2. Random Forest

```
rfc = RandomForestClassifier()
rfc.fit(X_train, Y_train)
Y_pred_rff = rfc.predict(X_test)
print('Accuracy %2.2f%%' % (100*accuracy_score(Y_test, Y_pred_rff)))
print(classification_report(Y_test, Y_pred_rff))
```

Clase	Precision	Recall	F1-Score	Support
1	0.57	0.71	0.63	17
2	0.50	0.36	0.42	14
Accuracy			0.55	31
Macro avg	0.54	0.53	0.52	31
Weighted avg	0.54	0.55	0.53	31

Cuadro 3: Métricas de rendimiento del modelo

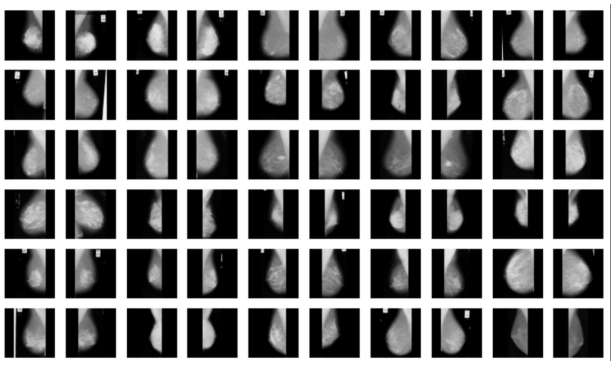
4. Conclusión

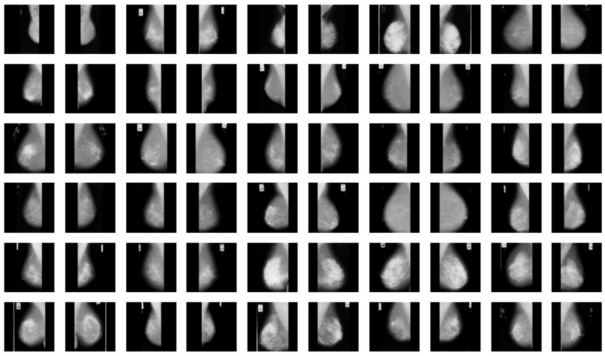
5. Anexo

5.1. Sección 1

```
import os
import math
```

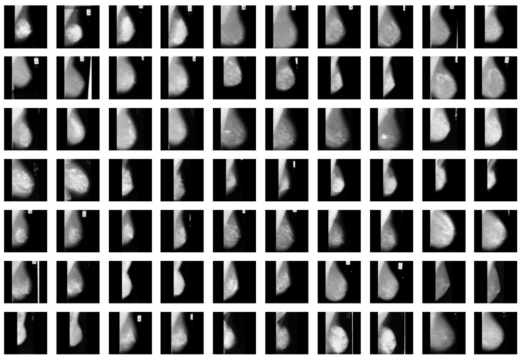
```
from skimage.segmentation import flood as flood
import h5py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy as sp
from scipy.stats import skew
from scipy.stats import kurtosis
import scipy.ndimage as ndi
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn import svm
from skimage.measure   import shannon_entropy
import cv2
from skimage import io
from skimage import color
from skimage.draw import polygon
#from skimage.feature import greycomatrix, greycoprops
from sklearn.decomposition import PCA


from skimage.feature import canny
from skimage.filters import sobel
from skimage.transform import hough_line, hough_line_peaks


from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

from IPython.display import Image
#Lectura de im genes
carpetas=[]
for i in range(0,120):
    if i<9:
      carpeta = cv2.imread(f'/content/drive/MyDrive/PAByB/TP␣final␣ima genes/mdb00{i+1}.pgm')
      carpetas.append(carpeta)
    elif i<99:
      carpeta = cv2.imread(f'/content/drive/MyDrive/PAByB/TP␣final␣ima genes/mdb0{i+1}.pgm')
      carpetas.append(carpeta)
    else:
      carpeta = cv2.imread(f'/content/drive/MyDrive/PAByB/TP␣final␣ima genes/mdb{i+1}.pgm')
      carpetas.append(carpeta)

plt.imshow(carpetas[1], cmap="gray")
```

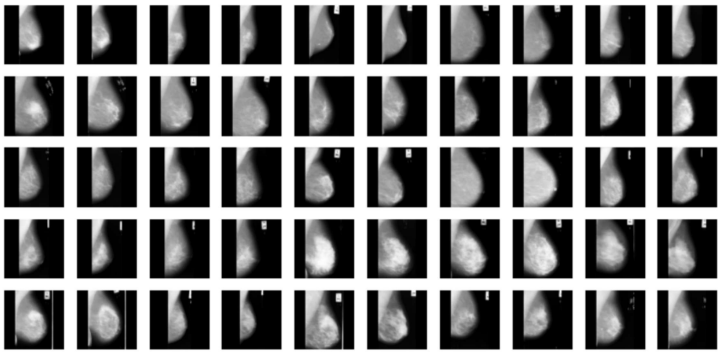




5.2. Sección 2

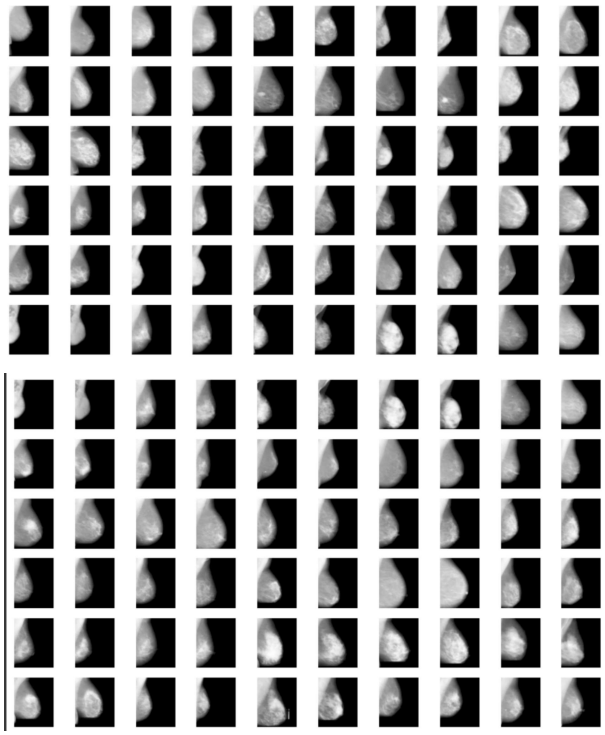
Los resultados de las 120 imágenes volteadas después del algortimos de volteo automático y manual:





5.3. Sección 3

Los resultados de las 120 imágenes volteadas después del algortimos de remoción de artefactos y recorte:



5.4. Sección 4

Las estadísticas de las 120 imágenes:

1. Imagen: 1, varianza: 0.0, media: 0.0
2. Imagen: 2, varianza: 0.0, media: 0.0
3. Imagen: 3, varianza: 0.0, media: 0.0
4. Imagen: 4, varianza: 0.0, media: 0.0
5. Imagen: 5, varianza: 0.0, media: 0.0
6. Imagen: 6, varianza: 0.0, media: 0.0
7. Imagen: 7, varianza: 0.0, media: 0.0
8. Imagen: 8, varianza: 0.0, media: 0.0
9. Imagen: 9, varianza: 0.0, media: 0.0
10. Imagen: 10, varianza: 0.0, media: 0.0
11. Imagen: 11, varianza: 0.0, media: 0.0
12. Imagen: 12, varianza: 0.0, media: 0.0
13. Imagen: 13, varianza: 0.0, media: 0.0
14. Imagen: 14, varianza: 0.0, media: 0.0
15. Imagen: 15, varianza: 0.0, media: 0.0
16. Imagen: 16, varianza: 0.0, media: 0.0
17. Imagen: 17, varianza: 0.0, media: 0.0
18. Imagen: 18, varianza: 0.0, media: 0.0

19. Imagen: 19, varianza: 0.0, media: 0.0
20. Imagen: 20, varianza: 0.0, media: 0.0
21. Imagen: 21, varianza: 0.0, media: 0.0
22. Imagen: 22, varianza: 0.0, media: 0.0
23. Imagen: 23, varianza: 0.0, media: 0.0
24. Imagen: 24, varianza: 0.0, media: 0.0
25. Imagen: 25, varianza: 0.0, media: 0.0
26. Imagen: 26, varianza: 0.0, media: 0.0
27. Imagen: 27, varianza: 0.0, media: 0.0
28. Imagen: 28, varianza: 0.0, media: 0.0
29. Imagen: 29, varianza: 0.0, media: 0.0
30. Imagen: 30, varianza: 0.0, media: 0.0
31. Imagen: 31, varianza: 0.0, media: 0.0
32. Imagen: 32, varianza: 0.0, media: 0.0
33. Imagen: 33, varianza: 0.0, media: 0.0
34. Imagen: 34, varianza: 0.0, media: 0.0
35. Imagen: 35, varianza: 0.0, media: 0.0
36. Imagen: 36, varianza: 0.0, media: 0.0
37. Imagen: 37, varianza: 0.0, media: 0.0
38. Imagen: 38, varianza: 0.0, media: 0.0
39. Imagen: 39, varianza: 0.0, media: 0.0
40. Imagen: 40, varianza: 0.0, media: 0.0
41. Imagen: 41, varianza: 0.0, media: 0.0
42. Imagen: 42, varianza: 0.0, media: 0.0
43. Imagen: 43, varianza: 0.0, media: 0.0
44. Imagen: 44, varianza: 0.0, media: 0.0
45. Imagen: 45, varianza: 0.0, media: 0.0
46. Imagen: 46, varianza: 0.0, media: 0.0
47. Imagen: 47, varianza: 0.0, media: 0.0
48. Imagen: 48, varianza: 0.0, media: 0.0
49. Imagen: 49, varianza: 0.0, media: 0.0
50. Imagen: 50, varianza: 0.0, media: 0.0
51. Imagen: 51, varianza: 0.0, media: 0.0
52. Imagen: 52, varianza: 0.0, media: 0.0
53. Imagen: 53, varianza: 0.0, media: 0.0
54. Imagen: 54, varianza: 0.0, media: 0.0
55. Imagen: 55, varianza: 0.0, media: 0.0

56. Imagen: 56, varianza: 0.0, media: 0.0
57. Imagen: 57, varianza: 0.0, media: 0.0
58. Imagen: 58, varianza: 0.0, media: 0.0
59. Imagen: 59, varianza: 0.0, media: 0.0
60. Imagen: 60, varianza: 0.0, media: 0.0
61. Imagen: 61, varianza: 0.0, media: 0.0
62. Imagen: 62, varianza: 0.0, media: 0.0
63. Imagen: 63, varianza: 0.0, media: 0.0
64. Imagen: 64, varianza: 0.0, media: 0.0
65. Imagen: 65, varianza: 0.0, media: 0.0
66. Imagen: 66, varianza: 0.0, media: 0.0
67. Imagen: 67, varianza: 0.0, media: 0.0
68. Imagen: 68, varianza: 0.0, media: 0.0
69. Imagen: 69, varianza: 0.0, media: 0.0
70. Imagen: 70, varianza: 0.0, media: 0.0
71. Imagen: 71, varianza: 0.0, media: 0.0
72. Imagen: 72, varianza: 0.0, media: 0.0
73. Imagen: 73, varianza: 0.0, media: 0.0
74. Imagen: 74, varianza: 0.0, media: 0.0
75. Imagen: 75, varianza: 0.0, media: 0.0
76. Imagen: 76, varianza: 0.0, media: 0.0
77. Imagen: 77, varianza: 0.0, media: 0.0
78. Imagen: 78, varianza: 0.0, media: 0.0
79. Imagen: 79, varianza: 0.0, media: 0.0
80. Imagen: 80, varianza: 0.0, media: 0.0
81. Imagen: 81, varianza: 0.0, media: 0.0
82. Imagen: 82, varianza: 0.0, media: 0.0
83. Imagen: 83, varianza: 0.0, media: 0.0
84. Imagen: 84, varianza: 0.0, media: 0.0
85. Imagen: 85, varianza: 0.0, media: 0.0
86. Imagen: 86, varianza: 0.0, media: 0.0
87. Imagen: 87, varianza: 0.0, media: 0.0
88. Imagen: 88, varianza: 0.0, media: 0.0
89. Imagen: 89, varianza: 0.0, media: 0.0
90. Imagen: 90, varianza: 0.0, media: 0.0
91. Imagen: 91, varianza: 0.0, media: 0.0
92. Imagen: 92, varianza: 0.0, media: 0.0

93. Imagen: 93, varianza: 0.0, media: 0.0
94. Imagen: 94, varianza: 0.0, media: 0.0
95. Imagen: 95, varianza: 0.0, media: 0.0
96. Imagen: 96, varianza: 0.0, media: 0.0
97. Imagen: 97, varianza: 0.0, media: 0.0
98. Imagen: 98, varianza: 0.0, media: 0.0
99. Imagen: 99, varianza: 0.0, media: 0.0
100. Imagen: 100, varianza: 0.0, media: 0.0
101. Imagen: 101, varianza: 0.0, media: 0.0
102. Imagen: 102, varianza: 0.0, media: 0.0
103. Imagen: 103, varianza: 0.0, media: 0.0
104. Imagen: 104, varianza: 0.0, media: 0.0
105. Imagen: 105, varianza: 0.0, media: 0.0
106. Imagen: 106, varianza: 0.0, media: 0.0
107. Imagen: 107, varianza: 0.0, media: 0.0
108. Imagen: 108, varianza: 0.0, media: 0.0
109. Imagen: 109, varianza: 0.0, media: 0.0
110. Imagen: 110, varianza: 0.0, media: 0.0
111. Imagen: 111, varianza: 0.0, media: 0.0
112. Imagen: 112, varianza: 0.0, media: 0.0
113. Imagen: 113, varianza: 0.0, media: 0.0
114. Imagen: 114, varianza: 0.0, media: 0.0
115. Imagen: 115, varianza: 0.0, media: 0.0
116. Imagen: 116, varianza: 0.0, media: 0.0
117. Imagen: 117, varianza: 0.0, media: 0.0
118. Imagen: 118, varianza: 0.0, media: 0.0
119. Imagen: 119, varianza: 0.0, media: 0.0
120. Imagen: 120, varianza: 0.0, media: 0.0

article enumerate

1. Imagen: 1, varianza: 0.0, media: 0.0
2. Imagen: 2, varianza: 0.0, media: 0.0
3. Imagen: 3, varianza: 0.0, media: 0.0
4. Imagen: 4, varianza: 0.0, media: 0.0
5. Imagen: 5, varianza: 0.0, media: 0.0
6. Imagen: 6, varianza: 0.0, media: 0.0
7. Imagen: 7, varianza: 0.0, media: 0.0
8. Imagen: 8, varianza: 0.0, media: 0.0

9. Imagen: 9, varianza: 0.0, media: 0.0
10. Imagen: 10, varianza: 0.0, media: 0.0
11. Imagen: 11, varianza: 0.0, media: 0.0
12. Imagen: 12, varianza: 0.0, media: 0.0
13. Imagen: 13, varianza: 0.0, media: 0.0
14. Imagen: 14, varianza: 0.0, media: 0.0
15. Imagen: 15, varianza: 0.0, media: 0.0
16. Imagen: 16, varianza: 0.0, media: 0.0
17. Imagen: 17, varianza: 0.0, media: 0.0
18. Imagen: 18, varianza: 0.0, media: 0.0
19. Imagen: 19, varianza: 0.0, media: 0.0
20. Imagen: 20, varianza: 0.0, media: 0.0
21. Imagen: 21, varianza: 0.0, media: 0.0
22. Imagen: 22, varianza: 0.0, media: 0.0
23. Imagen: 23, varianza: 0.0, media: 0.0
24. Imagen: 24, varianza: 0.0, media: 0.0
25. Imagen: 25, varianza: 0.0, media: 0.0
26. Imagen: 26, varianza: 0.0, media: 0.0
27. Imagen: 27, varianza: 0.0, media: 0.0
28. Imagen: 28, varianza: 0.0, media: 0.0
29. Imagen: 29, varianza: 0.0, media: 0.0
30. Imagen: 30, varianza: 0.0, media: 0.0
31. Imagen: 31, varianza: 0.0, media: 0.0
32. Imagen: 32, varianza: 0.0, media: 0.0
33. Imagen: 33, varianza: 0.0, media: 0.0
34. Imagen: 34, varianza: 0.0, media: 0.0
35. Imagen: 35, varianza: 0.0, media: 0.0
36. Imagen: 36, varianza: 0.0, media: 0.0
37. Imagen: 37, varianza: 0.0, media: 0.0
38. Imagen: 38, varianza: 0.0, media: 0.0
39. Imagen: 39, varianza: 0.0, media: 0.0
40. Imagen: 40, varianza: 0.0, media: 0.0
41. Imagen: 41, varianza: 0.0, media: 0.0
42. Imagen: 42, varianza: 0.0, media: 0.0
43. Imagen: 43, varianza: 0.0, media: 0.0
44. Imagen: 44, varianza: 0.0, media: 0.0
45. Imagen: 45, varianza: 0.0, media: 0.0

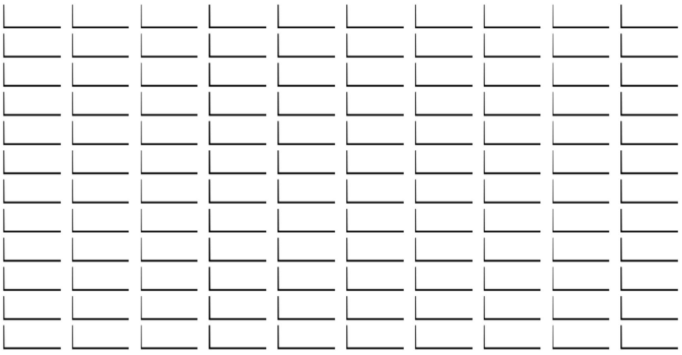
46. Imagen: 46, varianza: 0.0, media: 0.0
47. Imagen: 47, varianza: 0.0, media: 0.0
48. Imagen: 48, varianza: 0.0, media: 0.0
49. Imagen: 49, varianza: 0.0, media: 0.0
50. Imagen: 50, varianza: 0.0, media: 0.0
51. Imagen: 51, varianza: 0.0, media: 0.0
52. Imagen: 52, varianza: 0.0, media: 0.0
53. Imagen: 53, varianza: 0.0, media: 0.0
54. Imagen: 54, varianza: 0.0, media: 0.0
55. Imagen: 55, varianza: 0.0, media: 0.0
56. Imagen: 56, varianza: 0.0, media: 0.0
57. Imagen: 57, varianza: 0.0, media: 0.0
58. Imagen: 58, varianza: 0.0, media: 0.0
59. Imagen: 59, varianza: 0.0, media: 0.0
60. Imagen: 60, varianza: 0.0, media: 0.0
61. Imagen: 61, varianza: 0.0, media: 0.0
62. Imagen: 62, varianza: 0.0, media: 0.0
63. Imagen: 63, varianza: 0.0, media: 0.0
64. Imagen: 64, varianza: 0.0, media: 0.0
65. Imagen: 65, varianza: 0.0, media: 0.0
66. Imagen: 66, varianza: 0.0, media: 0.0
67. Imagen: 67, varianza: 0.0, media: 0.0
68. Imagen: 68, varianza: 0.0, media: 0.0
69. Imagen: 69, varianza: 0.0, media: 0.0
70. Imagen: 70, varianza: 0.0, media: 0.0
71. Imagen: 71, varianza: 0.0, media: 0.0
72. Imagen: 72, varianza: 0.0, media: 0.0
73. Imagen: 73, varianza: 0.0, media: 0.0
74. Imagen: 74, varianza: 0.0, media: 0.0
75. Imagen: 75, varianza: 0.0, media: 0.0
76. Imagen: 76, varianza: 0.0, media: 0.0
77. Imagen: 77, varianza: 0.0, media: 0.0
78. Imagen: 78, varianza: 0.0, media: 0.0
79. Imagen: 79, varianza: 0.0, media: 0.0
80. Imagen: 80, varianza: 0.0, media: 0.0
81. Imagen: 81, varianza: 0.0, media: 0.0
82. Imagen: 82, varianza: 0.0, media: 0.0

83. Imagen: 83, varianza: 0.0, media: 0.0
84. Imagen: 84, varianza: 0.0, media: 0.0
85. Imagen: 85, varianza: 0.0, media: 0.0
86. Imagen: 86, varianza: 0.0, media: 0.0
87. Imagen: 87, varianza: 0.0, media: 0.0
88. Imagen: 88, varianza: 0.0, media: 0.0
89. Imagen: 89, varianza: 0.0, media: 0.0
90. Imagen: 90, varianza: 0.0, media: 0.0
91. Imagen: 91, varianza: 0.0, media: 0.0
92. Imagen: 92, varianza: 0.0, media: 0.0
93. Imagen: 93, varianza: 0.0, media: 0.0
94. Imagen: 94, varianza: 0.0, media: 0.0
95. Imagen: 95, varianza: 0.0, media: 0.0
96. Imagen: 96, varianza: 0.0, media: 0.0
97. Imagen: 97, varianza: 0.0, media: 0.0
98. Imagen: 98, varianza: 0.0, media: 0.0
99. Imagen: 99, varianza: 0.0, media: 0.0
100. Imagen: 100, varianza: 0.0, media: 0.0
101. Imagen: 101, varianza: 0.0, media: 0.0
102. Imagen: 102, varianza: 0.0, media: 0.0
103. Imagen: 103, varianza: 0.0, media: 0.0
104. Imagen: 104, varianza: 0.0, media: 0.0
105. Imagen: 105, varianza: 0.0, media: 0.0
106. Imagen: 106, varianza: 0.0, media: 0.0
107. Imagen: 107, varianza: 0.0, media: 0.0
108. Imagen: 108, varianza: 0.0, media: 0.0
109. Imagen: 109, varianza: 0.0, media: 0.0
110. Imagen: 110, varianza: 0.0, media: 0.0
111. Imagen: 111, varianza: 0.0, media: 0.0
112. Imagen: 112, varianza: 0.0, media: 0.0
113. Imagen: 113, varianza: 0.0, media: 0.0
114. Imagen: 114, varianza: 0.0, media: 0.0
115. Imagen: 115, varianza: 0.0, media: 0.0
116. Imagen: 116, varianza: 0.0, media: 0.0
117. Imagen: 117, varianza: 0.0, media: 0.0
118. Imagen: 118, varianza: 0.0, media: 0.0
119. Imagen: 119, varianza: 0.0, media: 0.0

120. Imagen: 120, varianza: 0.0, media: 0.0

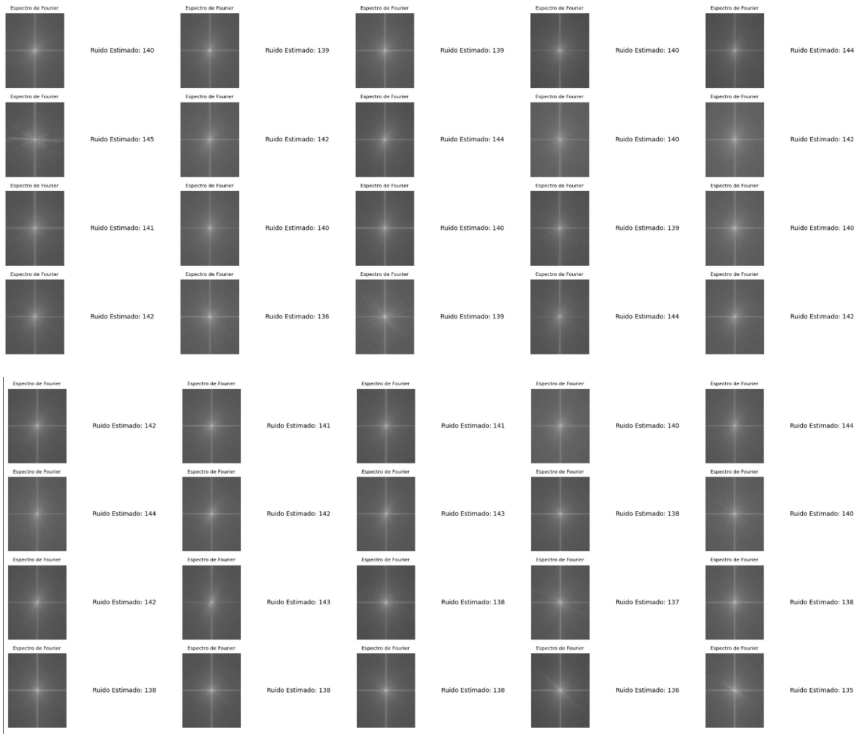
5.5. Sección 5

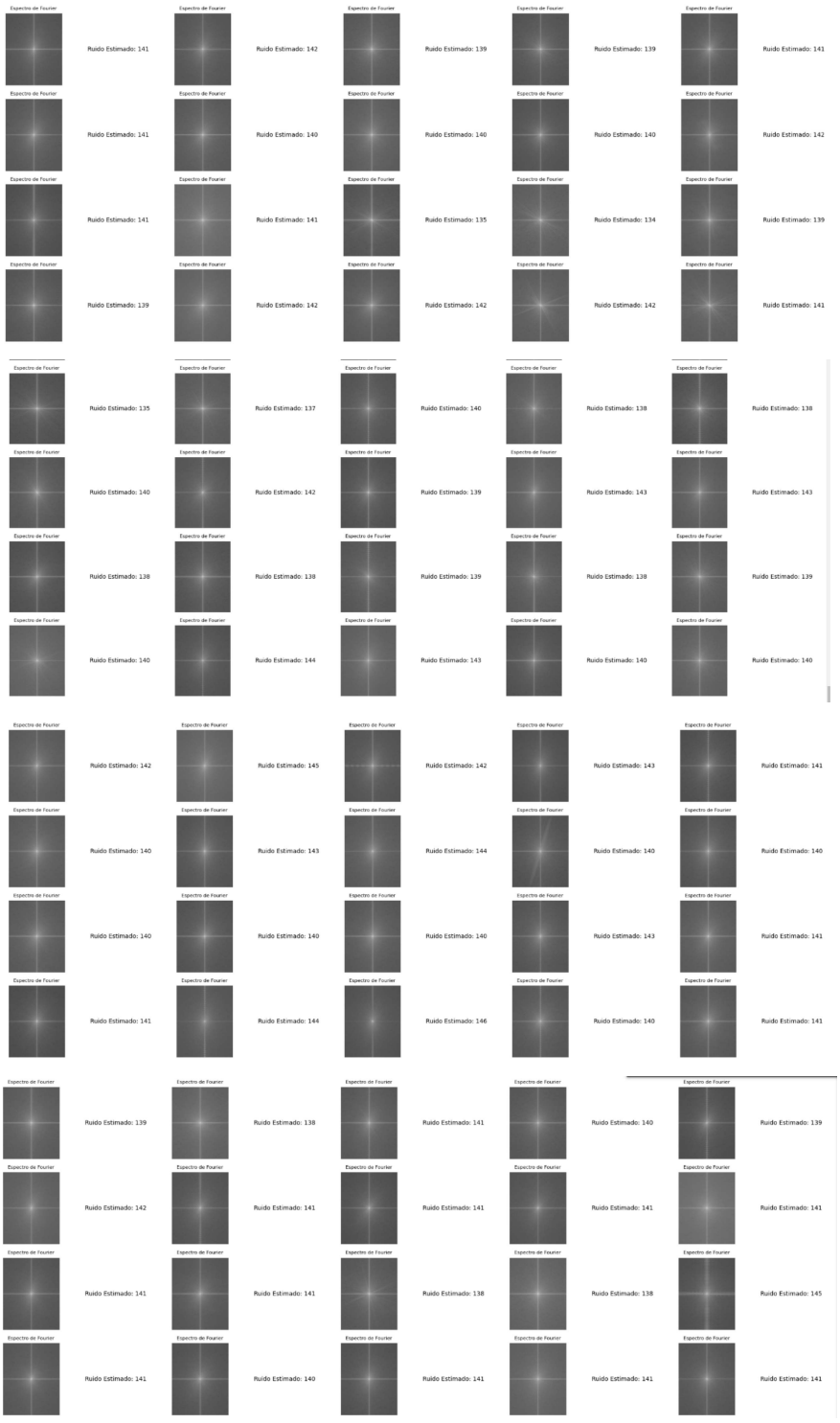
Los histogramas de las 120 imágenes:



5.6. Sección 6

Estimar el ruido de las 120 imágenes con la transfromada de fourier:





5.7. Sección 7

Estimación del ruido para las 120 imágenes con la transformada wavelet:

1. Imagen número: 1 - Estimación del nivel de ruido: 1.8689807653427124
2. Imagen número: 2 - Estimación del nivel de ruido: 1.802535057067871

3. Imagen número: 3 - Estimación del nivel de ruido: 1.8874021768569946
4. Imagen número: 4 - Estimación del nivel de ruido: 2.0666253566741943
5. Imagen número: 5 - Estimación del nivel de ruido: 2.4214725494384766
6. Imagen número: 6 - Estimación del nivel de ruido: 3.074233055114746
7. Imagen número: 7 - Estimación del nivel de ruido: 2.2409183979034424
8. Imagen número: 8 - Estimación del nivel de ruido: 2.6394755840301514
9. Imagen número: 9 - Estimación del nivel de ruido: 2.126112937927246
10. Imagen número: 10 - Estimación del nivel de ruido: 2.2248759269714355
11. Imagen número: 11 - Estimación del nivel de ruido: 2.234548330307007
12. Imagen número: 12 - Estimación del nivel de ruido: 2.115666627883911
13. Imagen número: 13 - Estimación del nivel de ruido: 2.0661253929138184
14. Imagen número: 14 - Estimación del nivel de ruido: 1.8446522951126099
15. Imagen número: 15 - Estimación del nivel de ruido: 2.5071961879730225
16. Imagen número: 16 - Estimación del nivel de ruido: 2.13735294342041
17. Imagen número: 17 - Estimación del nivel de ruido: 1.9961323738098145
18. Imagen número: 18 - Estimación del nivel de ruido: 1.7712582349777222
19. Imagen número: 19 - Estimación del nivel de ruido: 4.070398330688477
20. Imagen número: 20 - Estimación del nivel de ruido: 2.12402606010437
21. Imagen número: 21 - Estimación del nivel de ruido: 2.667194128036499
22. Imagen número: 22 - Estimación del nivel de ruido: 2.0977752208709717
23. Imagen número: 23 - Estimación del nivel de ruido: 2.7396633625030518
24. Imagen número: 24 - Estimación del nivel de ruido: 1.9767929315567017
25. Imagen número: 25 - Estimación del nivel de ruido: 2.3265624046325684
26. Imagen número: 26 - Estimación del nivel de ruido: 2.506366014480591
27. Imagen número: 27 - Estimación del nivel de ruido: 2.4394209384918213
28. Imagen número: 28 - Estimación del nivel de ruido: 2.08256459236145
29. Imagen número: 29 - Estimación del nivel de ruido: 2.207547664642334
30. Imagen número: 30 - Estimación del nivel de ruido: 1.990684151649475
31. Imagen número: 31 - Estimación del nivel de ruido: 2.1412241458892822
32. Imagen número: 32 - Estimación del nivel de ruido: 2.6578333377838135
33. Imagen número: 33 - Estimación del nivel de ruido: 2.0014541149139404
34. Imagen número: 34 - Estimación del nivel de ruido: 1.9445034265518188
35. Imagen número: 35 - Estimación del nivel de ruido: 1.8432408571243286
36. Imagen número: 36 - Estimación del nivel de ruido: 1.825203537940979
37. Imagen número: 37 - Estimación del nivel de ruido: 2.127866268157959
38. Imagen número: 38 - Estimación del nivel de ruido: 1.8153581619262695
39. Imagen número: 39 - Estimación del nivel de ruido: 1.7432446479797363

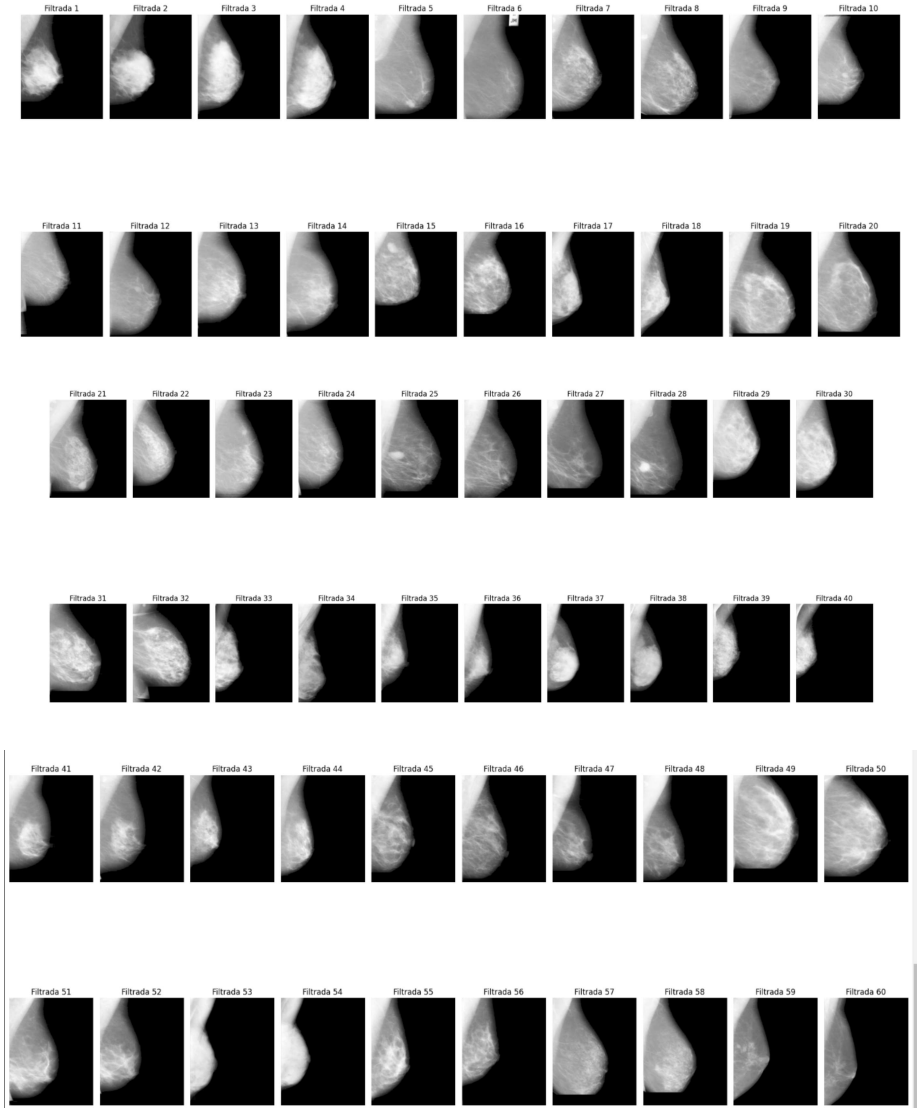
40. Imagen número: 40 - Estimación del nivel de ruido: 1.7187353372573853
41. Imagen número: 41 - Estimación del nivel de ruido: 2.2822506427764893
42. Imagen número: 42 - Estimación del nivel de ruido: 1.9695792198181152
43. Imagen número: 43 - Estimación del nivel de ruido: 1.7710380554199219
44. Imagen número: 44 - Estimación del nivel de ruido: 1.9147462844848633
45. Imagen número: 45 - Estimación del nivel de ruido: 2.0536909103393555
46. Imagen número: 46 - Estimación del nivel de ruido: 2.014547109603882
47. Imagen número: 47 - Estimación del nivel de ruido: 2.3213260173797607
48. Imagen número: 48 - Estimación del nivel de ruido: 1.988856315612793
49. Imagen número: 49 - Estimación del nivel de ruido: 2.4313416481018066
50. Imagen número: 50 - Estimación del nivel de ruido: 2.1680057048797607
51. Imagen número: 51 - Estimación del nivel de ruido: 2.301175117492676
52. Imagen número: 52 - Estimación del nivel de ruido: 2.024676561355591
53. Imagen número: 53 - Estimación del nivel de ruido: 2.0260775089263916
54. Imagen número: 54 - Estimación del nivel de ruido: 1.740662932395935
55. Imagen número: 55 - Estimación del nivel de ruido: 1.9878877401351929
56. Imagen número: 56 - Estimación del nivel de ruido: 1.8398040533065796
57. Imagen número: 57 - Estimación del nivel de ruido: 2.1044318675994873
58. Imagen número: 58 - Estimación del nivel de ruido: 2.110410690307617
59. Imagen número: 59 - Estimación del nivel de ruido: 2.0633766651153564
60. Imagen número: 60 - Estimación del nivel de ruido: 2.0635201930999756
61. Imagen número: 61 - Estimación del nivel de ruido: 1.6553648710250854
62. Imagen número: 62 - Estimación del nivel de ruido: 1.6961761713027954
63. Imagen número: 63 - Estimación del nivel de ruido: 2.167093515396118
64. Imagen número: 64 - Estimación del nivel de ruido: 1.9893079996109009
65. Imagen número: 65 - Estimación del nivel de ruido: 1.971517562866211
66. Imagen número: 66 - Estimación del nivel de ruido: 1.8222814798355103
67. Imagen número: 67 - Estimación del nivel de ruido: 2.9530441761016846
68. Imagen número: 68 - Estimación del nivel de ruido: 1.9184436798095703
69. Imagen número: 69 - Estimación del nivel de ruido: 2.5601069927215576
70. Imagen número: 70 - Estimación del nivel de ruido: 2.308398962020874
71. Imagen número: 71 - Estimación del nivel de ruido: 2.1295979022979736
72. Imagen número: 72 - Estimación del nivel de ruido: 1.8300957679748535
73. Imagen número: 73 - Estimación del nivel de ruido: 2.3841640949249268
74. Imagen número: 74 - Estimación del nivel de ruido: 2.045501947402954
75. Imagen número: 75 - Estimación del nivel de ruido: 1.848164677619934
76. Imagen número: 76 - Estimación del nivel de ruido: 2.033203363418579

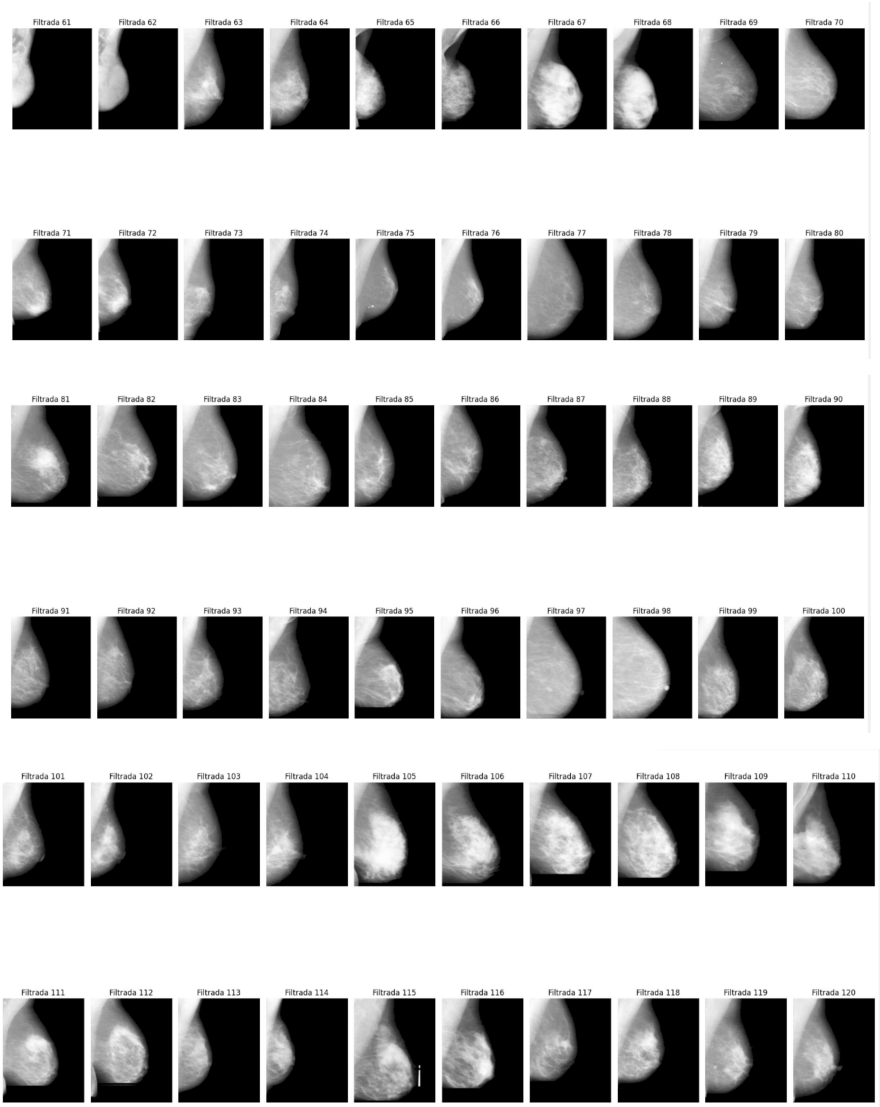
77. Imagen número: 77 - Estimación del nivel de ruido: 2.3889577388763428
78. Imagen número: 78 - Estimación del nivel de ruido: 2.1368560791015625
79. Imagen número: 79 - Estimación del nivel de ruido: 1.9782567024230957
80. Imagen número: 80 - Estimación del nivel de ruido: 2.006558418273926
81. Imagen número: 81 - Estimación del nivel de ruido: 2.3163273334503174
82. Imagen número: 82 - Estimación del nivel de ruido: 2.3216593265533447
83. Imagen número: 83 - Estimación del nivel de ruido: 2.3052332401275635
84. Imagen número: 84 - Estimación del nivel de ruido: 3.0524799823760986
85. Imagen número: 85 - Estimación del nivel de ruido: 2.354590654373169
86. Imagen número: 86 - Estimación del nivel de ruido: 1.8919099569320679
87. Imagen número: 87 - Estimación del nivel de ruido: 2.220329523086548
88. Imagen número: 88 - Estimación del nivel de ruido: 2.365697145462036
89. Imagen número: 89 - Estimación del nivel de ruido: 1.98248291015625
90. Imagen número: 90 - Estimación del nivel de ruido: 2.026078224182129
91. Imagen número: 91 - Estimación del nivel de ruido: 1.9340434074401855
92. Imagen número: 92 - Estimación del nivel de ruido: 1.9067901372909546
93. Imagen número: 93 - Estimación del nivel de ruido: 2.0456106662750244
94. Imagen número: 94 - Estimación del nivel de ruido: 2.1941678524017334
95. Imagen número: 95 - Estimación del nivel de ruido: 2.466956853866577
96. Imagen número: 96 - Estimación del nivel de ruido: 2.0937047004699707
97. Imagen número: 97 - Estimación del nivel de ruido: 3.742621660232544
98. Imagen número: 98 - Estimación del nivel de ruido: 2.978510856628418
99. Imagen número: 99 - Estimación del nivel de ruido: 2.5331292152404785
100. Imagen número: 100 - Estimación del nivel de ruido: 2.115816116333008
101. Imagen número: 101 - Estimación del nivel de ruido: 1.9253875017166138
102. Imagen número: 102 - Estimación del nivel de ruido: 1.8380743265151978
103. Imagen número: 103 - Estimación del nivel de ruido: 2.264296770095825
104. Imagen número: 104 - Estimación del nivel de ruido: 2.027695417404175
105. Imagen número: 105 - Estimación del nivel de ruido: 2.309577465057373
106. Imagen número: 106 - Estimación del nivel de ruido: 2.0446033477783203
107. Imagen número: 107 - Estimación del nivel de ruido: 3.1517837047576904
108. Imagen número: 108 - Estimación del nivel de ruido: 1.98811674118042
109. Imagen número: 109 - Estimación del nivel de ruido: 2.381478786468506
110. Imagen número: 110 - Estimación del nivel de ruido: 2.0648281574249268
111. Imagen número: 111 - Estimación del nivel de ruido: 2.113908290863037
112. Imagen número: 112 - Estimación del nivel de ruido: 2.5404443740844727
113. Imagen número: 113 - Estimación del nivel de ruido: 2.0609776973724365

114. Imagen número: 114 - Estimación del nivel de ruido: 1.839152216911316
115. Imagen número: 115 - Estimación del nivel de ruido: 3.500232696533203
116. Imagen número: 116 - Estimación del nivel de ruido: 2.4591991901397705
117. Imagen número: 117 - Estimación del nivel de ruido: 2.058652639389038
118. Imagen número: 118 - Estimación del nivel de ruido: 2.029325485229492
119. Imagen número: 119 - Estimación del nivel de ruido: 2.1040849685668945
120. Imagen número: 120 - Estimación del nivel de ruido: 2.068614959716797

5.8. Sección 8

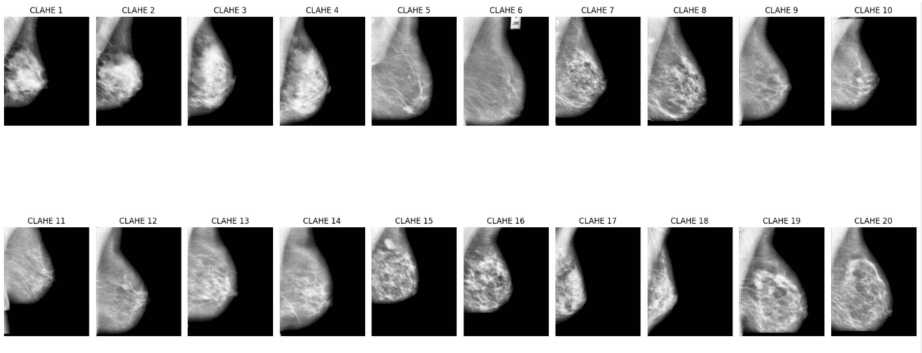
Los resultados de los filtros promediadores de las 120 imágenes:

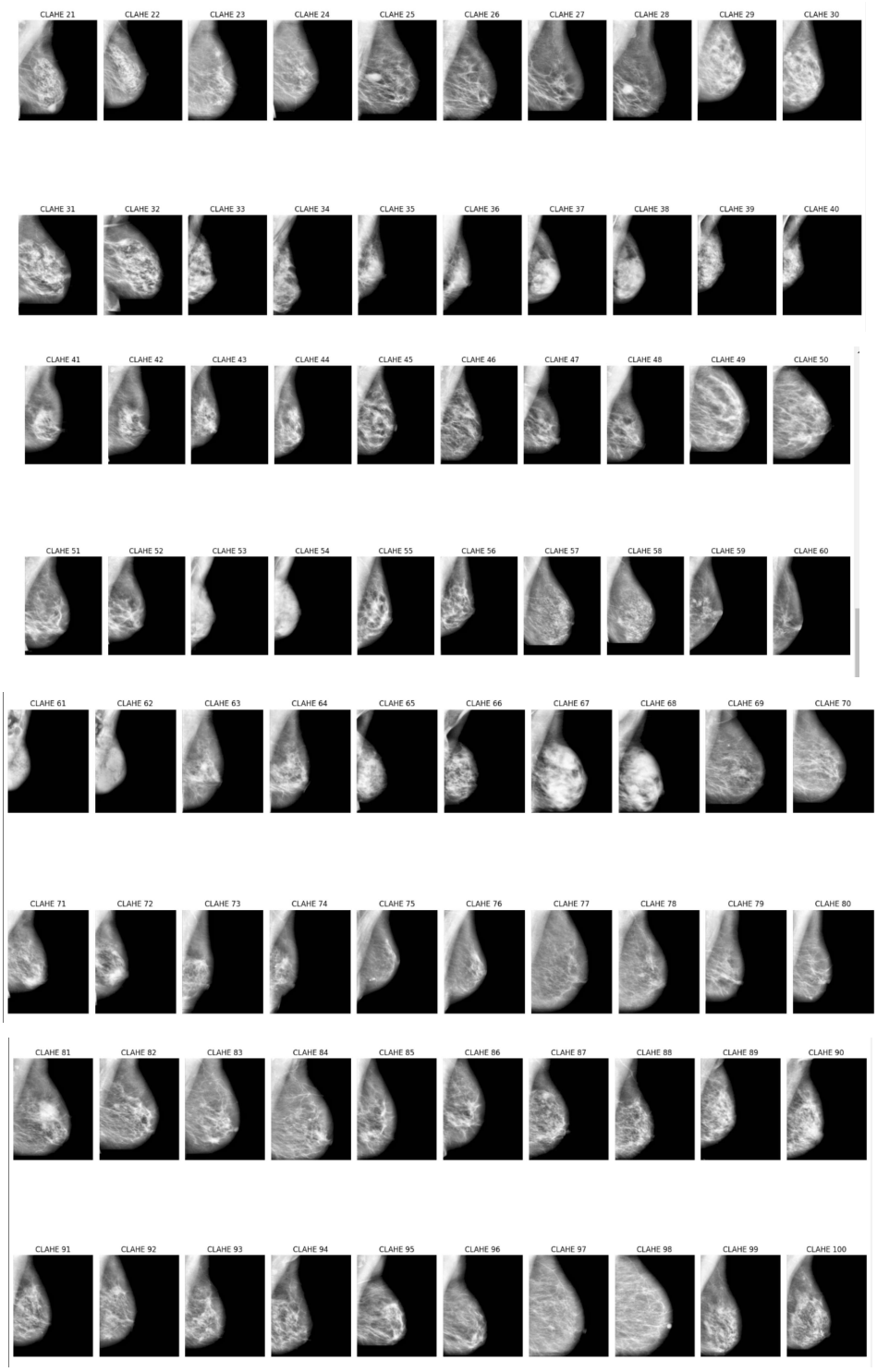


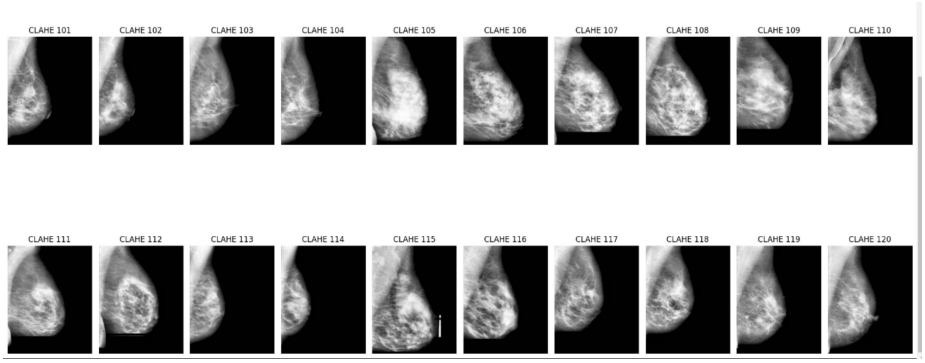


5.9. Sección 9

Los resultados de de las 120 imágenes con CLAHE:




CLAHE 21
CLAHE 22
CLAHE 23
CLAHE 24
CLAHE 25
CLAHE 26
CLAHE 27
CLAHE 28
CLAHE 29
CLAHE 30
CLAHE 31
CLAHE 32
CLAHE 33
CLAHE 34
CLAHE 35
CLAHE 36
CLAHE 37
CLAHE 38
CLAHE 39
CLAHE 40
CLAHE 41
CLAHE 42
CLAHE 43
CLAHE 44
CLAHE 45
CLAHE 46
CLAHE 47
CLAHE 48
CLAHE 49
CLAHE 50
CLAHE 51
CLAHE 52
CLAHE 53
CLAHE 54
CLAHE 55
CLAHE 56
CLAHE 57
CLAHE 58
CLAHE 59
CLAHE 60
CLAHE 61
CLAHE 62
CLAHE 63
CLAHE 64
CLAHE 65
CLAHE 66
CLAHE 67
CLAHE 68
CLAHE 69
CLAHE 70
CLAHE 71
CLAHE 72
CLAHE 73
CLAHE 74
CLAHE 75
CLAHE 76
CLAHE 77
CLAHE 78
CLAHE 79
CLAHE 80
CLAHE 81
CLAHE 82
CLAHE 83
CLAHE 84
CLAHE 85
CLAHE 86
CLAHE 87
CLAHE 88
CLAHE 89
CLAHE 90
CLAHE 91
CLAHE 92
CLAHE 93
CLAHE 94
CLAHE 95
CLAHE 96
CLAHE 97
CLAHE 98
CLAHE 99
CLAHE 100


CLAHE 101
CLAHE 102
CLAHE 103
CLAHE 104
CLAHE 105
CLAHE 106
CLAHE 107
CLAHE 108
CLAHE 109
CLAHE 110
CLAHE 111
CLAHE 112
CLAHE 113
CLAHE 114
CLAHE 115
CLAHE 116
CLAHE 117
CLAHE 118
CLAHE 119
CLAHE 120